

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики – процессов управления

Шаго Павел Евгеньевич

Выпускная квалификационная работа

**«Математическое моделирование и реализация
планировщика процессов для гетерогенных
процессорных архитектур»**

Уровень образования: бакалавриат

Направление: 01.03.02 Прикладная математика и информатика

Основная образовательная программа СВ.5005.2022: «Прикладная математика,
фундаментальная информатика и программирование»

Научный руководитель
кандидат физ.-мат. наук, доцент
Корхов Владимир Владиславович

Рецензент
Эксперт по архитектуре и сетевому стеку
ООО НИЦ АЭРОДИСК
Анохин Юрий Владимирович

Санкт-Петербург
2026

Содержание

Введение	3
Постановка задачи	4
1 Подходы к реализации планировщиков в ОС	5
1.1 Задача планирования	5
1.2 Гетерогенные процессорные архитектуры	6
1.3 Расширяемый планировщик Linux	7
1.4 Актуальные исследования планировщиков	10
1.5 Анализ подходов к реализации планировщика	13
2 Математические модели планировщиков	15
2.1 Введение	15
2.2 Обозначения и допущения	15
2.3 Earliest Eligible Virtual Deadline First	18
2.4 Аукционная модель A1349	21
3 Реализация алгоритмов планирования	28
3.1 Реализация EEVDF с помощью sched_ext	28
3.2 Реализация аукционной модели A1349	31
4 Описание тестового окружения и экспериментального оборудования	40
4.1 Тестовое оборудование и программное окружение	40
4.2 Набор бенчмарков	41
4.3 Измеряемые метрики	42
5 Анализ результатов эксперимента	44
5.1 Результаты при лёгкой нагрузке	44
5.2 Результаты при стрессовой нагрузке	45
5.3 Результаты при умеренной нагрузке	47
5.4 Выводы	48
Заключение	50
Список литературы	51

Введение

В современных операционных системах механизмы многозадачности являются фундаментальным инструментом управления вычислительными ресурсами. Ключевую роль в этих механизмах играет планировщик, отвечающий за распределение процессорного времени между потоками исполнения. От качества его решений напрямую зависят пропускная способность системы под нагрузкой, время отклика интерактивных приложений и энергопотребление оборудования.

Параллельно с развитием операционных систем меняется и аппаратная платформа. Долгое время многопроцессорные системы строились по симметричному принципу, при котором все ядра одного процессора обладают одинаковой производительностью. В современных процессорах этот принцип постепенно вытесняется гетерогенным подходом, при котором на одном кристалле сочетаются производительные и энергоэффективные ядра. Подобная организация применяется в мобильных процессорах ARM big.LITTLE, в настольных и серверных процессорах Intel начиная с поколения Alder Lake, а в последние годы появляется и в экосистеме RISC-V. В таких системах выбор ядра напрямую влияет на время выполнения задачи и её энергопотребление, поэтому планировщик не может игнорировать различия ядер при размещении задач.

Интерактивные, пакетные и фоновые нагрузки предъявляют к планировщику свои требования, и оптимальная для одного сценария политика редко оказывается удачной для другого. Возникает потребность подбирать политику планирования под конкретный класс задач, однако ядро Linux остаётся общим для всех пользователей, а сопровождение собственной модификации штатного планировщика ради узкой задачи требует значительных усилий и плохо переносимо между версиями ядра.

В ответ на этот запрос в ядро Linux был включён фреймворк `sched_ext`, позволяющий реализовывать политику планирования как программу, загружаемую в ядро во время работы системы без его пересборки. Это существенно ускоряет цикл разработки и снижает порог для прототипирования новых алгоритмов планирования.

В настоящей работе строится математическая модель планировщика, учитывающая различие ядер по вычислительной мощности, и на её основе предлагается аукционная модель A1349 как альтернатива базовому планировщику Linux. Средствами `sched_ext` реализованы обе модели, A1349 и базовый алгоритм EEVDF. Реализация EEVDF позволяет дополнительно оценить накладные расходы самого фреймворка. Полученные реализации сравниваются со штатным планировщиком на наборе тестов при различных типах нагрузок.

Постановка задачи

Объектом исследования являются алгоритмы планирования задач (процессов) в операционных системах на вычислительных устройствах с гетерогенной процессорной архитектурой (Intel Hybrid, ARM big.LITTLE и другие).

Целью работы является разработка и экспериментальная оценка планировщика, обеспечивающего более высокую эффективность вычислений на гетерогенном оборудовании по сравнению с базовым алгоритмом EEVDF ядра Linux.

Для достижения цели работы необходимо:

1. Проанализировать и определить наиболее перспективный подход к реализации планировщика.
2. Построить математическую модель, учитывающую различную вычислительную мощность ядер процессора.
3. Реализовать предложенный алгоритм с использованием инфраструктуры `sched_ext` и разработать воспроизводимый набор бенчмарков.
4. Провести эксперимент на гетерогенном оборудовании и оценить результаты в сравнении с базовым EEVDF.

1 Подходы к реализации планировщиков в ОС

1.1 Задача планирования

В современных операционных системах механизмы многозадачности являются фундаментальным инструментом управления вычислительными ресурсами. Реализация концепции псевдопараллелизма базируется на процедурах контекстного переключения, которые позволяют распределять кванты процессорного времени между потоками исполнения.

Ключевым вызовом в данном контексте является задача построения эффективного расписания, при котором порядок выполнения задач минимизирует простои оборудования и максимизирует целевые показатели эффективности. Обычно задачу построения такого расписания и дальнейшего распределения задач по ядрам процессора решает компонент непосредственно встроенный в ядро операционной системы называемый планировщиком.

С развитием операционных систем и оборудования планировщики также эволюционировали. Так, в операционной системе UNIX планировщик выбирал задачу по приоритету, а среди равных задач использовался алгоритм похожий на round-robin [1]. Позднее в ядре Linux версии 2.4 был реализован планировщик $O(n)$, который при планировании следующей задачи просматривал очередь готовых к исполнению задач и вычислял для них эвристику *goodness* [2]. В Linux 2.6 его сменил планировщик $O(1)$, в котором разделили очередь приоритетов на два массива, *active* и *expired*, что позволило выбирать следующую задачу за постоянное время [3].

Затем в версии Linux 2.6.23 произошёл переход к планировщику Completely Fair Scheduler (CFS), основанному на модели виртуального времени. Готовые к выполнению задачи хранились в красно-чёрном дереве, упорядоченном по виртуальному времени, а планировщик выбирал задачу, получившую меньше всего процессорного времени относительно своей справедливой доли [4]. В Linux 6.6 эта идея получила развитие в планировщике Earliest Eligible Virtual Deadline First (EEVDF), где вводятся отставание и виртуальный дедлайн. Допустимыми считаются задачи, у которых отставание $\text{lag} \geq 0$, среди них выбирается задача с ближайшим виртуальным дедлайном [5].

Таким образом, за последние полвека планировщик операционной системы прошёл путь от детерминированных приоритетных схем, близких по логике к round-robin, до сложных алгоритмов динамического распределения ресурсов с сильной математической базой.

1.2 Гетерогенные процессорные архитектуры

Параллельно с развитием операционных систем меняется и аппаратная платформа, на которой они исполняются. Если классические многопроцессорные системы строились на симметричном принципе (SMP), при котором все ядра обладают одинаковой производительностью и одинаковым энергопотреблением, то современные процессоры всё чаще объединяют на одном кристалле ядра разных классов. Архитектура ARM big.LITTLE впервые сделала такой подход массовым в мобильном сегменте, объединив производительные ядра (*big*) с энергоэффективными (*LITTLE*) [6]. В настольных и серверных процессорах Intel начиная с поколения Alder Lake внедрена гибридная архитектура с Performance и Efficient ядрами (P/E) [7]. Принцип не привязан к конкретной системе команд, аналогичные конфигурации прорабатываются и в экосистеме RISC-V, где появляются SoC, объединяющие на одном кристалле ядра разных классов производительности, например, гетерогенный SoC Marsellus [8].

Ключевая особенность подобных систем заключается в том, что ядра одного процессора отличаются по тактовой частоте, ширине внеочередного исполнения, размеру кэшей и удельному энергопотреблению. Одна и та же задача, выполненная на P-ядре, завершается быстрее, но обходится дороже по энергии, тогда как E-ядро обеспечивает лучшее соотношение производительность/ватт при более низкой пропускной способности. В подобных системах выбор ядра для исполнения задачи существенно влияет на итоговое поведение системы. Размещение чувствительной к задержкам задачи на E-ядре увеличивает время её обслуживания, а выполнение фоновой нагрузки на P-ядре приводит к избыточному энергопотреблению.

В ядре Linux различия между ядрами обрабатываются подсистемой Capacity Aware Scheduling (CAS), которая распознаёт неоднородные топологии и учитывает относительную мощность каждого ядра при балансировке. Поверх неё работает Energy Aware Scheduling (EAS), которая по модели энергопотребления и оценкам загрузки ядер выбирает для задачи ядро процессора с меньшим потреблением без потери пропускной способности. EAS включается лишь при выполнении ряда условий, среди которых наличие энергомоделей, регулятор частоты schedutil и отсутствие SMT, поэтому применяется прежде всего на мобильных SoC семейства ARM big.LITTLE [9]. На стороне x86 для информирования планировщика о приоритете ядер задействуется механизм Intel Thread Director, транслирующий аппаратные подсказки о характере нагрузки в теги класса задачи [7]. Однако CAS и Thread Director закрывают лишь балансировку по вычислительной мощности и классификацию по профилю инструкций, тогда как отдельные очереди готовых задач для P/E-кластеров, приоритизация чувствительных к задержкам нагрузок и прикладные правила выбора кластера остаются за пределами штатной подсистемы. Реализация подобных политик внутри fair-класса требует существенных изменений ядра, что мотивирует переход к расширяемым механизмам, рассматриваемым в следующем разделе.

1.3 Расширяемый планировщик Linux

`sched_ext` (scheduler extensions) [10] – инфраструктура (фреймворк) ядра Linux, позволяющая реализовывать политику планирования как eBPF-программу, подключаемую к интерфейсу расширяемых планировщиков во время работы системы и не требующую пересборки ядра. Фреймворк `sched_ext` был включён в основную ветку Linux в версии 6.12 и предоставляет отдельный класс планирования `ext_sched_class`, расположенный между классами `fair_sched_class` и `idle_sched_class`. При загруженном eBPF-планировщике обычные задачи переводятся из `fair`-класса под управление `sched_ext` (см. рис. 1.1).

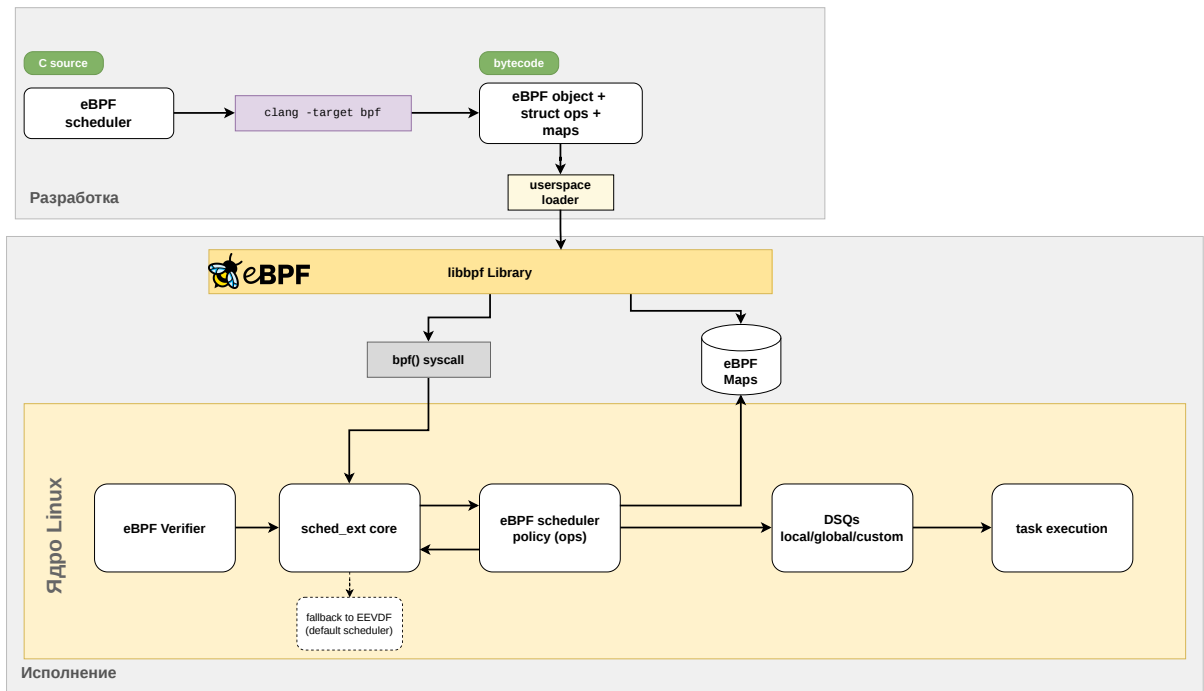


Рис. 1.1: Архитектура `sched_ext` в ядре Linux

Подход с вынесением политики планирования из ядра встречался и до `sched_ext`. В системе `ghOST` [11] политика запускается как пользовательский процесс-агент, а ядро ОС исполняет полученные от агента решения о переключениях. Агент видит состояние задач через разделяемую память и отвечает за выбор задачи и процессорного ядра, тогда как ядро ОС берёт на себя лишь переключение контекста, что позволяет описывать политику на обычном языке программирования и опираться на пользовательские средства отладки.

Важно отметить, что `sched_ext` замещает только политику справедливого планирования (`fair`-класс). Задачи реального времени (`SCHED_FIFO`, `SCHED_RR`) и задачи с дедлайнами (`SCHED_DEADLINE`) остаются под управлением штатных классов `rt_sched_class` и `dl_sched_class` соответственно. Это ограничивает область применимости пользовательских политик, но одновременно сохраняет гарантии для критичных по времени задач.

При штатной выгрузке `sched_ext` планировщика, при наличии в нем критической ошибки или при срабатывании специального таймера ядро автоматически возвращает все задачи под управление `EEVDF`, что обеспечивает безопасность экспериментов с пользовательскими политиками непосредственно на работающей системе.

С точки зрения программной модели, политика планирования в `sched_ext` описывается набором обратных вызовов, объединённых в структуру `struct sched_ext_ops`. Основными обработчиками являются: `select_cpu`, вызываемый при пробуждении задачи и отвечающий за выбор ядра исполнения, `enqueue`, помещающий задачу в очередь готовых к выполнению, `dispatch`, выбирающий следующую задачу для конкретного ядра, а также `running` и `stopping`, дающие дополнительные контрольные точки для обновления метрик (при пробуждении и снятии задачи). Дополнительные обработчики `init_task`, `exit_task`, `cpu_online` и `cpu_offline` позволяют поддерживать собственное состояние, связанное с задачами и ядрами, на протяжении всего их жизненного цикла. Отдельная группа обработчиков `cgroup_init`, `cgroup_exit`, `cgroup_move` и `cgroup_set_weight` обеспечивает интеграцию с иерархией контрольных групп: они вызываются при создании и удалении `cgroup`, перемещении задач между группами, а также при изменении веса группы, что позволяет реализовывать иерархические политики планирования.

Поведение фреймворка настраивается через флаги поля `ops->flags`. Флаг `SCX_OPS_SWITCH_PARTIAL` переводит под управление `sched_ext` только задачи с явной политикой `SCHED_EXT`, оставляя `SCHED_NORMAL`, `SCHED_BATCH` и `SCHED_IDLE` в `fair`-классе. Флаг `SCX_OPS_ENQ_LAST` определяет, попадает ли последняя выполнявшаяся задача обратно в очередь после исчерпания кванта. Флаг `SCX_OPS_KEEP_BUILTIN_IDLE` указывает ядру продолжать обновлять битовую маску простаивающих CPU при загруженной пользовательской политике. Без этого флага eBPF-программе пришлось бы вести учёт самостоятельно, тогда как с ним поиск свободного ядра доступен через штатные `kfunc`-вызовы.

Передача задач между обработчиками происходит через очереди диспетчеризации *dispatch queue* (DSQ). Помимо встроенных глобальной `SCX_DSQ_GLOBAL` и локальных по числу ядер `SCX_DSQ_LOCAL`, eBPF-программа может создавать произвольное количество собственных очередей с заданным идентификатором при помощи `scx_bpf_create_dsq`. Каждая DSQ обслуживается по принципу FIFO либо по виртуальному времени, задаваемому при вставке через `scx_bpf_dsq_insert_vtime`. В режиме виртуального времени очередь реализована как красно-чёрное дерево, упорядоченное по полю `vtime`, что обеспечивает извлечение задачи с наименьшим виртуальным временем за $O(\log n)$. Возможность создавать произвольные DSQ позволяет завести отдельную очередь на каждый кластер ядер и обслуживать её по собственному виртуальному времени. Тогда задача, помещённая в очередь нужного кластера на этапе `enqueue`, попадёт на исполнение только к ядрам этого кластера, а упорядочивание по `vtime` сохраняется внутри кластера независимо от остальных.

Такое разделение существенно в контексте гетерогенных архитектур, где Р/Е-ядра отличаются по производительности и требуют отдельного учёта.

Для взаимодействия с ядром eBPF-программа использует набор *kfunc*-вызовов, образующих стабильный интерфейс планировщика: `scx_bpf_dsq_insert` помещает задачу в выбранную очередь, `scx_bpf_dsq_move_to_local` извлекает задачу из очереди для исполнения на текущем ядре. Состояние задач и ядер хранится в `BPF_MAP`-структурах, а доступ к полям `task_struct` и `rq` осуществляется через механизм CO-RE (Compile Once – Run Everywhere), что обеспечивает переносимость скомпилированной программы между версиями ядра.

Для типичных нужд балансировки фреймворк предоставляет встроенный учёт простаивающих ядер: функции `scx_bpf_pick_idle_cpu` и `scx_bpf_test_and_clear_cpu_idle` позволяют без собственной реализации находить свободные CPU по битовой маске. Если обработчик `select_cpu` помещает задачу непосредственно в локальную очередь `SCX_DSQ_LOCAL` целевого ядра, последующий вызов `enqueue` для этой задачи пропускается, что образует быстрый путь пробуждения и сокращает накладные расходы на часто пробуждающихся задачах.

Собственное состояние, привязанное к конкретной задаче, удобно хранить в структуре типа `BPF_MAP_TYPE_TASK_STORAGE`, которая обеспечивает константное время доступа без хеш-поиска и автоматически освобождается при завершении задачи. Длительность кванта процессорного времени задаётся присваиванием поля `p->scx.slice` в обработчиках `enqueue` или `dispatch`, а значением по умолчанию служит `SCX_SLICE_DFL` (20 мс). Вытеснение уже исполняющейся задачи на удалённом ядре инициируется вызовом `scx_bpf_kick_cpu` с флагом `SCX_KICK_PREEMPT`.

Безопасность исполнения произвольной политики обеспечивается несколькими механизмами времени выполнения. Таймер `watchdog` отслеживает случаи, когда задача не получает процессорного времени дольше установленного порога (по умолчанию 30 секунд), и при срабатывании выгружает eBPF-планировщик. Дополнительно реализована возможность экстренного отключения пользовательской политики комбинацией клавиш `SysRq-S`.

На этапе загрузки программы среда eBPF накладывает на политику планирования ряд архитектурных ограничений, проверяемых верификатором. В программе запрещены блокирующие вызовы и динамическая аллокация памяти, любой цикл должен иметь верифицируемую верхнюю границу, а доступ к структурам ядра разрешён только через утверждённый набор *kfunc* и полей. Эти требования заметно сказываются на выборе структур данных и способов агрегирования метрик при разработке планировщика.

1.4 Актуальные исследования планировщиков

С появлением фреймворка `sched_ext` [10] исследовательская активность в области процессных планировщиков заметно возросла. Большинство современных работ используют `sched_ext` как экспериментальную платформу для быстрой проверки новых алгоритмов без модификации ядра. Диапазон применения охватывает оптимизации существующих алгоритмов, подходы машинного обучения и нестандартные постановки задач.

Tang et al. [12] предложили `sched_ext`-планировщик SRAND, в котором расчёт отставаний и виртуальных дедлайнов EEVDF заменён собственной упрощённой схемой виртуального времени и многоуровневой очередью. По завершении кванта виртуальное время задачи увеличивается на величину $(slice - p.slice) \cdot 100 / p.weight$, где $slice = 20$ мс — длительность кванта, $p.slice$ — его неиспользованный остаток, $p.weight$ — вес задачи. У задач с большим весом виртуальное время растёт медленнее, что соответствует более высокому приоритету. Только что появившиеся и пробуждающиеся задачи инициализируются текущим глобальным виртуальным временем системы, которое монотонно подтягивается вверх по максимуму активных задач. Если у проснувшейся задачи виртуальное время меньше $V(t) - slice$, оно принудительно поднимается до этого порога, что не даёт долго спавшей задаче монополизировать ядро.

Полученное виртуальное время служит ключом распределения по пяти BPF-очередям с порогами 20, 50, 200 и 500 мс (`queue0–queue4`). Очереди с меньшим индексом опустошаются первыми и попадают в единую глобальную FIFO-очередь, откуда ядра напрямую извлекают следующую задачу, что заменяет обход красно-чёрного дерева EEVDF одной операцией взятия с головы. Дополнительно поддерживаются локальные очереди свободных ядер процессора с привязкой задач к ядрам, на которых они уже работали, а потоки ядра получают неограниченный квант и направляются непосредственно в локальную очередь. Политика реализована через обработчики `select_cpu`, `enqueue` и `dispatch`, время ожидания задачи в очереди ограничено порогом 5000 мс. По данным авторов, на Lmbench время переключения контекста сокращается на 11,83%, а на haskbench общая производительность растёт на 7,02% при сопоставимой с EEVDF равномерности распределения нагрузки (вариативность времени работы ядер 0,005 против 0,006 у EEVDF).

Xu et al. [13] применили `sched_ext` к задаче управляемого тестирования параллелизма (*controlled concurrency testing*, CCT) ядра Linux. Планировщик LACE сериализует конфликтующие потоки и переключает их в контролируемом порядке, автоматически вставляя точки переключения в операциях с разделяемой памятью и примитивах синхронизации, что позволяет систематически воспроизводить состояния гонки без вмешательства гипервизора. По сравнению с фаззером SegFuzz LACE достигает на 38% большего покрытия ветвлений, снижает накладные расходы на 57% и ускоряет воспроизведение ошибок в 11,4 раза, обнаружив восемь ранее неизвестных ошибок параллелизма при объёме патчей ядра менее 25 строк. Работа показывает, что `sched_ext`

пригоден не только для оптимизации производительности, но и как инструмент верификации корректности параллельного кода.

Мао et al. [14] рассмотрели задачу планирования с точки зрения обеспечения полноты данных о происхождении (*provenance*). При высоком темпе генерации событий EEVDF, LAVD и Rusty теряют от 39% до более чем 98% событий, что компрометирует гарантии эталонного монитора и открывает возможность для угрозы суперпродюсера. Планировщик Aegis строится на двухслойной архитектуре. Первый слой делит задачи на основную и несколько непервичных очередей: для непервичных задаётся время ожидания, играющее роль бюджета ресурсов, а выбор очереди происходит по наиболее *голодной* из них, что обеспечивает справедливость без явных приоритетов. Второй слой управляет переключением через DeepQ-Network с двойным вознаграждением — r_c штрафует за потерю событий и обеспечивает полноту, r_p поощряет утилизацию процессора. На бенчмарках с системами Sysdig и eAudit Aegis показал нулевую потерю событий при суммарных накладных расходах менее 0,5% в типичных режимах и около 2,44% в наиболее тяжёлых, что достигается дельта-функцией пропуска несущественных решений планирования.

Wang et al. [15] развили идею адаптивного планирования до уровня портфеля политик. Предложенный агент Adaptive Scheduling Agent (ASA) декомпозирует задачу на распознавание шаблона нагрузки и сопоставление ему планировщика из набора предобученных экспертов. Шаблон определяется ансамблевым классификатором на основе XGBoost по метрикам утилизации процессора, ввода-вывода и сетевой активности, собираемым через eBPF и `procfs`, а для подавления шумов применяется взвешенное по времени вероятностное голосование с экспоненциальным затуханием. Подготовка агента ведётся в три этапа: прототипное обучение базовой модели, динамическая калибровка стоимости переключения и обучение модели обобщения. Динамическое переключение `sched_ext`-политики выполняется на лету. ASA превосходит EEVDF в 86,4% тестовых сценариев и попадает в тройку лучших политик в 78,9% случаев, а на смешанных нагрузках обгоняет даже статический оракул за счёт переключения политик внутри одной задачи. Агент потребляет около 1,45% одного ядра и 297 МБ памяти и сохраняет качество при переносе на новые аппаратные платформы, не входившие в обучающую выборку. Работа подчёркивает ограниченность статичной политики при росте разнообразия нагрузок и оборудования.

Galin и Hedblom [16] оценили планировщик LAVD (Latency-criticality Aware Virtual Deadline) в telco-окружении Kubernetes Minikube. Поводом к работе послужила проблема задержек в облачной инфраструктуре Ericsson, резко возрастающих при загрузке процессора выше 50%, что делает оценку поведения планировщика на интервале 50–95% критически важной. LAVD изначально проектировался для игровых нагрузок и предлагается для telco в силу схожих требований к задержке. Для воспроизводимой оценки авторы реализовали многопоточный симулятор на C++, генерирующий нагрузки на основе трассировочных данных Ericsson: распределение времени обслуживания подобрано как бета-распределение, давшее наибольшее значение $p = 0,084$ по критерию Колмогорова–Смирнова, плюс отдельный генератор фоновой интерферирующей нагрузки.

В исследовании сопоставлены три планировщика: LAVD, минималистичный `scx_simple` без поддержки вытеснения и штатный EEVDF в роли базовой линии. При высокой доле фоновой интерферирующей нагрузки LAVD сокращает среднее время оборота относительно EEVDF на 6–81% и улучшает 99-й перцентиль до 90%. Однако в типичных для Ericsson условиях (50% загрузки трафиком и 10% интерферирующей нагрузки) `scx_simple` стабильно обгоняет LAVD, несмотря на значительно меньшую сложность, а попытки тонкой настройки минимального и максимального кванта LAVD не дают значимых улучшений. Авторы делают вывод, что преимущество принадлежит не отдельной политике, а самой платформе `sched_ext` как средству быстрого сравнения алгоритмов и адаптации планирования к доменной специфике.

Перечисленные работы охватывают оптимизацию очередей, тестирование параллелизма, обучаемые политики и доменную адаптацию, но исходят из однородного вычислительного ресурса. Задача планирования на гетерогенных процессорах в рамках `sched_ext` остаётся значительно менее изученной. Ближе всего к ней стоит работа Van Craeynest et al. [17], в которой интенсивность обращений к памяти признана недостаточным критерием сопоставления задачи типу ядра, а оценка производительности выводится из совместного учёта ILP и MLP на уровне CPI-компонент. Предложенный механизм PIE прогнозирует поведение задачи на альтернативном ядре по CPI-стеку без её миграции и даёт прирост пропускной способности на 5,5% над современными планировщиками и на 8,7% над политиками на основе сэмплирования. Однако работа относится к 2012 году и предлагает аппаратный механизм оценки, тогда как в настоящей работе тот же учёт ведётся программно в рамках `sched_ext`. Заполнение этого пробела — учёт вычислительной мощности ядер в программной политике планирования на современных гетерогенных платформах и составляет предмет настоящей работы.

1.5 Анализ подходов к реализации планировщика

Способ реализации планировщика определяет темп исследования, цену ошибки в политике и переносимость результатов между версиями ядра. К настоящему моменту в практике закрепились четыре подхода: прямая модификация штатного fair-класса, загружаемый модуль ядра, пользовательский агент по схеме ghOSt и eBPF-программа в рамках sched_ext.

Прямая модификация штатного fair-класса обеспечивает наилучшее быстродействие, поскольку политика выполняется без промежуточных слоёв и имеет полный доступ к внутренним структурам ядра. Однако цикл разработки оказывается дорогим. Каждое изменение требует пересборки ядра (порядка 10 минут на современной машине) и перезагрузки целевой системы, ошибка в логике планирования с высокой вероятностью приводит к панике ядра, зависанию или повреждению состояния системы, а сравнение нескольких вариантов превращается в линейное накопление времени сборки и перезагрузки. Подход оправдан при подготовке итоговой реализации к включению в основную ветку Linux, но плохо подходит для итеративного исследования.

Загружаемый модуль ядра сокращает цикл итерации до секунд, поскольку вместо пересборки ядра достаточно перезагрузки модуля. Однако последствия ошибок остаются прежними, модуль исполняется с привилегиями ядра, и некорректная политика так же способна привести к панике, зависанию или повреждению состояния системы, как и встроенная реализация. Кроме того, в Linux отсутствует официальный интерфейс для замены планировщика fair-класса, поэтому модуль вынужден опираться на внутренние структуры ядра, такие как `struct sched_class` и поля `task_struct`, регулярно меняющиеся между версиями. ABI ядра в общем случае не стабилизирован, и решение оказывается жёстко привязано к конкретной версии ядра и требует сопровождения при каждом обновлении.

Пользовательский агент по схеме ghOSt [11] переносит логику планирования в обычный процесс, что обеспечивает устойчивость к ошибкам политики и широкий выбор языков реализации. Взаимодействие с ядром построено на разделяемой памяти и транзакционной передаче решений, поэтому накладные расходы заметно ниже наивной модели на системных вызовах, однако каждое решение всё равно требует синхронизации между ядром и агентом и проигрывает реализации внутри ядра на нагрузках с короткими и частыми пробуждениями. Помимо этого, ghOSt поставляется отдельным набором патчей ядра, не входящим в основную ветку, поэтому требует сопровождения при каждом обновлении ядра.

eBPF-программа в рамках sched_ext снимает основные ограничения предыдущих подходов. Программа выполняется внутри ядра, обеспечивая сопоставимое со встроенной реализацией быстродействие. Безопасность гарантируется верификатором eBPF на этапе загрузки (см. раздел 1.3), а таймер watchdog при зависании политики автоматически выгружает eBPF-планировщик и возвращает систему под управление EEVDF. Подключение и выгрузка планировщика выполняются на работающей системе

за миллисекунды без перезагрузки, а механизм CO-RE обеспечивает переносимость скомпилированной программы между близкими версиями ядра. Существенно и то, что с момента принятия в основную ветку `sched_ext` стал фактическим стандартом сравнения новых алгоритмов планирования, что упрощает воспроизведение и прямое сопоставление результатов с существующими работами.

Результаты сравнения сведены в таблицу 1.1, где знаки $+$, $-$ и $\pm$ обозначают полное, отсутствующее и частичное выполнение критерия. Устойчивость к ошибкам политики здесь означает отсутствие риска паники, зависания или повреждения состояния ядра при некорректной логике планирования, а поддерживаемый интерфейс замены — наличие официального ABI для подключения внешнего планировщика без правки внутренних структур ядра.

Таблица 1.1: Сравнение подходов к реализации планировщиков

Критерий	В ядре	Модуль	ghOSt	<code>sched_ext</code>
Без пересборки ядра	—	+	$\pm$	+
Устойчивость к ошибкам политики	—	—	+	+
Низкие накладные расходы	+	+	—	$\pm$
Поддерживаемый интерфейс замены	—	—	$\pm$	+
Переносимость между версиями ядра	—	—	$\pm$	+

Среди рассмотренных подходов только `sched_ext` одновременно обеспечивает работу без пересборки ядра, устойчивость к ошибкам политики, поддерживаемый интерфейс замены и переносимость между версиями ядра, проигрывая встроенной реализации лишь по накладным расходам. Сочетание этих свойств делает возможным итеративное исследование на работающей системе и прямое сопоставление результатов с другими работами, поэтому в качестве основы для дальнейшей реализации выбран фреймворк `sched_ext`.

2 Математические модели планировщиков

2.1 Введение

Глава открывается сводкой обозначений и допущений, после чего описываются две модели планирования.

EEVDF [5] — алгоритм пропорционально- долевого планирования, лежащий в основе штатного планировщика Linux. Модель оперирует понятиями виртуального времени, отставания и виртуального дедлайна, обеспечивая доказуемо оптимальные границы справедливости при однородном вычислительном ресурсе. В настоящей работе она служит базовой моделью для сравнения.

Аукционная модель A1349 расширяет постановку задачи планирования на случай гетерогенного ресурса. Она опирается на динамический VCG-механизм распределения ресурсов [18, 19] и трактует выбор типа ядра как задачу аукционного распределения недолговечного блага между конкурирующими агентами. Гетерогенность ядер встроена в саму функцию ценности и в правило аллокации, а бюджеты ограничивают суммарный платёж задачи.

2.2 Обозначения и допущения

В таблицах 2.1–2.3 приведена сводка обозначений, используемых в настоящей и последующих главах.

Таблица 2.1: Обозначения модели EEVDF

Символ	Описание
t	Момент времени (непрерывный)
w_i	Вес (приоритет) клиента i
$\mathcal{A}(t)$	Множество активных клиентов (задач) в момент t
q	Верхняя граница длины запроса (квант процессорного времени)
$f_i(t)$	Идеальная мгновенная доля ресурса для клиента i
$S_i(t_0, t_1)$	Идеальное обслуживание клиента i на интервале $[t_0, t_1]$
$s_i(t_0, t_1)$	Фактически полученное обслуживание клиента i
t_0^i	Момент входа клиента i в систему
$\text{lag}_i(t)$	Отставание клиента i : $S_i - s_i$
$V(t)$	Виртуальное время системы
$r^{(k)}$	Длина запроса k (в единицах обслуживания)
$ve^{(k)}$	Виртуальное время доступности запроса k
$vd^{(k)}$	Виртуальный дедлайн запроса k
$u^{(k)}$	Фактически использованная часть запроса k
$r_{\max}^k$	Максимальная длина запроса клиента k

Таблица 2.2: Обозначения модели A1349. Платформа

Символ	Описание
$\mathcal{P}, \mathcal{E}$	Множества Р-ядер и Е-ядер
K_P, K_E	Число Р-ядер и Е-ядер
K	Общее число ядер, $K = K_P + K_E$
K_P^t, K_E^t	Число свободных ядер каждого типа в момент t
κ	Тип ядра, $\kappa \in \{P, E\}$
η_P, η_E	Производительность Р-ядра и Е-ядра
σ	Отношение производительностей, $\sigma = \eta_P/\eta_E > 1$
c_P, c_E	Базовая стоимость одного кванта на Р-ядре и Е-ядре
γ	Отношение стоимостей, $\gamma = c_P/c_E > 1$

Таблица 2.3: Обозначения модели A1349. Задачи и механизм

Символ	Описание
t	Момент времени (дискретный)
$\mathcal{N}^t$	Множество активных задач в момент t
$\theta_i = (v_i, l_i)$	Тип (характеристики) задачи i
v_i	Ценность завершения задачи i для пользователя
$\bar{V}$	Верхняя граница ценности задачи
l_i	Требуемое число квантов задачи i на Р-ядре, $l_i \in \{1, \dots, L\}$
L	Верхняя граница длины задачи на Р-ядре
$m_\kappa(l)$	Длина контракта задачи длины l на ядре типа $\kappa \in \{P, E\}$: $m_P(l) = l$, $m_E(l) = \lceil l\sigma \rceil$
B_i	Начальный бюджет задачи i
B_i^t	Остаточный бюджет в момент t : $B_i^t = B_i - \sum_{s < t} p_i^s$
χ_i	Класс обслуживания задачи i (real-time, interactive, batch, background)
s^t	Состояние MDP: $(\mathbf{x}^t, \mathbf{b}^t)$
$\mathbf{x}^t$	Остаточные длительности контрактов на ядрах
$\mathbf{b}^t$	Остаточные бюджеты активных задач
$a_i^t = (a_{i,P}^t, a_{i,E}^t)$	Назначение задачи i на тип ядра (компонент действия a^t)
$\phi_P(\theta_i), \phi_E(\theta_i)$	Эффективная ценность задачи на Р/Е-ядре, $\phi_\kappa = v_i - c_\kappa m_\kappa(l_i)$
$W(s^t)$	Социальная функция благосостояния (значение Беллмана)
$W_{-i}(s^t)$	Благосостояние без участия задачи i
δ	Коэффициент дисконтирования, $\delta \in (0, 1)$
p_i^t	VCG-платёж задачи i в момент t
π	Политика планирования
$u_i(\pi)$	Индивидуальная полезность задачи i при политике π (Допущение 9)
$\bar{W}_\kappa$	Ожидаемое будущее благосостояние для ядра типа κ

Далее перечислены допущения, общие для обеих моделей.

1. Все ядра одного типа считаются идентичными по производительности, различие существует только между Р/Е кластерами.
2. Зависимости между задачами по данным и синхронизации не учитываются моделью.

3. Планирование выполняется в онлайн-режиме, решение о назначении задачи принимается в момент её поступления без информации о будущих задачах.
4. Производительность ядер постоянна, η_P и η_E не изменяются во времени.
5. В каждый квант ядро исполняет ровно одну задачу, задача не может занимать несколько ядер одновременно.
6. Квант неделим, вытеснение происходит только на границе квантов.
7. Задача может исполняться на любом ядре (P или E), миграция между типами допускается только между квантами.
8. Контекстное переключение считается мгновенным, его стоимость не моделируется.
9. В модели A1349 задача получает полезность v_i только при полном завершении контракта длиной $m_\kappa(l_i)$ квантов, частичное исполнение полезности не приносит. Бюджетная допустимость задачи проверяется до аллокации, после чего контракт исполняется целиком и не прерывается до завершения.
10. Тип $\theta_i = (v_i, l_i)$ и класс χ_i задаются внешне и не сообщаются задачей. Задачи не стратегические. Формализм типа и аукциона используется только для правила аллокации и единицы бюджета.

2.3 Earliest Eligible Virtual Deadline First

Earliest Eligible Virtual Deadline First (EEVDF) [5] — модель пропорционально-долевого планирования, предложенная Ion Stoica и Hussein Abdel-Wahab.

Рассматривается один разделяемый ресурс (процессор), время на котором выделяется квантами длительности не более q . Множество клиентов (задач), активных в момент t , обозначается $\mathcal{A}(t)$, каждому клиенту i назначен положительный вес w_i . Идеальная мгновенная доля ресурса для клиента i определяется как

$$f_i(t) = \frac{w_i}{\sum_{j \in \mathcal{A}(t)} w_j}. \quad (2.1)$$

В идеализированной непрерывной модели при $q \rightarrow 0$ обслуживание $S_i(t_0, t_1)$, положенное клиенту i на интервале $[t_0, t_1]$, есть интеграл от f_i :

$$S_i(t_0, t_1) = \int_{t_0}^{t_1} f_i(\tau) d\tau. \quad (2.2)$$

Разность между положенным и фактически полученным обслуживанием задаёт центральную метрику справедливости, называемую отставанием (lag):

$$\text{lag}_i(t) = S_i(t_0^i, t) - s_i(t_0^i, t), \quad (2.3)$$

где t_0^i — момент входа клиента i в систему, s_i — фактически полученное обслуживание. Положительный lag соответствует недообслуженному клиенту, отрицательный — переобслуженному.

Виртуальное время системы $V(t)$ вводится как

$$V(t) = \int_0^t \frac{d\tau}{\sum_{j \in \mathcal{A}(\tau)} w_j}. \quad (2.4)$$

Пока клиент i активен на $[t_1, t_2]$, обслуживание линейно по виртуальному времени:

$$S_i(t_1, t_2) = w_i \cdot (V(t_2) - V(t_1)), \quad (2.5)$$

поскольку $\int_{t_1}^{t_2} w_i / W(\tau) d\tau = w_i (V(t_2) - V(t_1))$, где $W(\tau) = \sum_{j \in \mathcal{A}(\tau)} w_j$. За одну единицу виртуального времени каждый активный клиент получает ровно w_i единиц реального обслуживания.

Каждый запрос k клиента i описывается тремя величинами: длиной $r^{(k)}$, виртуальным временем доступности $ve^{(k)}$ и виртуальным дедлайном $vd^{(k)}$. Для краткости индекс (k) опускается, когда речь идёт об одном запросе. Запрос считается *доступным* (eligible) в момент t , если $ve \leq V(t)$.

Время доступности определяет момент, начиная с которого запрос может быть обслужен. Оно выбирается так, чтобы переобслуженный клиент ожидал, пока его идеальное обслуживание сравняется с фактически полученным:

$$ve = V(t_0^i) + \frac{s_i(t_0^i, t)}{w_i}. \quad (2.6)$$

При входе клиента i в момент t_0^i первый запрос получает $ve^{(1)} = V(t_0^i)$, что соответствует нулевому отставанию в момент входа.

Виртуальный дедлайн выбирается так, чтобы за интервал $[ve, vd]$ в непрерывной модели клиент получал ровно r единиц обслуживания:

$$vd = ve + \frac{r}{w_i}. \quad (2.7)$$

После исполнения запроса k с использованной частью $u^{(k)} \leq r^{(k)}$ доступность следующего запроса обновляется как $ve^{(k+1)} = ve^{(k)} + u^{(k)} / w_i$. В частном случае полного исполнения ($u^{(k)} = r^{(k)}$) получаем $ve^{(k+1)} = vd^{(k)}$.

Правило выбора очередной задачи состоит из двух стадий. Сначала применяется

фильтр доступности $ve \leq V(t)$, затем среди доступных запросов выбирается тот, у которого виртуальный дедлайн vd минимален. Алгоритм допускает реализацию с помощью дополненного двоичного дерева поиска со сложностью $O(\log n)$ на каждую операцию (выбор, вставку, удаление).

Лемма 1 (доступность недообслуженного клиента) [5]. Если $\text{lag}_k(t) \geq 0$ для активного клиента k , то k имеет доступный запрос в момент t .

Из $\text{lag}_k(t) \geq 0$ следует $s_k(t_0^k, t) \leq S_k(t_0^k, t)$. Тожество $S_k(t_0^k, t) = w_k(V(t) - V(t_0^k))$ сохраняется и при динамических событиях за счёт коррекций V , рассматриваемых ниже, откуда $ve = V(t_0^k) + s_k/w_k \leq V(t_0^k) + S_k/w_k = V(t)$.

Лемма 2 [5]. В любой момент t сумма отставаний всех активных клиентов равна нулю:

$$\sum_{i \in \mathcal{A}(t)} \text{lag}_i(t) = 0. \quad (2.8)$$

Следствие 1 [5]. Из Лемм 1 и 2 при непустом $\mathcal{A}(t)$ существует хотя бы один доступный запрос, поэтому EEVDF является work-conserving алгоритмом, то есть ресурс не простаивает при наличии активных клиентов.

При динамических событиях (вход, выход клиентов, изменение весов) виртуальное время системы корректируется для сохранения инварианта Леммы 2. При выходе клиента j в момент t

$$V(t^+) = V(t) + \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t^+)} w_i}, \quad (2.9)$$

при повторном входе клиента, сохранившего своё отставание $\text{lag}_j(t)$ с предыдущей сессии,

$$V(t^+) = V(t) - \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t^+)} w_i}. \quad (2.10)$$

При изменении веса клиента j с w_j на w'_j операция эквивалентна последовательности выход–вход с новым весом:

$$V(t^+) = V(t) + \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t)} w_i - w_j} - \frac{\text{lag}_j(t)}{\sum_{i \in \mathcal{A}(t)} w_i - w_j + w'_j}. \quad (2.11)$$

В [5] рассматриваются три стратегии обработки динамических событий, различающиеся тем, как поступать с отставанием клиента при входе и выходе. Первая сохраняет отставание и корректирует V по приведённым формулам, обеспечивая наилучшую долгосрочную справедливость. Вторая обнуляет отставание при выходе. Третья допускает события только при нулевом отставании, что гарантирует непрерывность V и упрощает анализ. Границы отставания, приведённые ниже, доказаны для третьей стратегии.

Для формулировки границ отставания вводится понятие стационарной системы [5].

Определение. Система называется стационарной, если все динамические со-

бытия (вход, выход, изменение веса) происходят при нулевом отставании участника.

В стационарной системе виртуальное время $V(t)$ непрерывно.

Лемма 3 (граница дедлайна) [5]. В стационарной системе любой запрос активного клиента k с дедлайном d обслуживается не позднее $d + q$, где q — размер кванта.

Теорема 1 (границы отставания) [5]. В стационарной системе для любого активного клиента k отставание ограничено как

$$-r_{\max} < \text{lag}_k(t) < \max(r_{\max}, q), \quad (2.12)$$

где $r_{\max}$ — максимальная длина запроса клиента k . Обе границы асимптотически точны.

Асимметрия объясняется природой EEVDF. Переобслуживание ($\text{lag} < 0$) ограничено снизу величиной $-r_{\max}$: после полного исполнения запроса виртуальное время доступности продвигается на r/w_k , и клиент теряет право на обслуживание до истечения собственного интервала. Недообслуживание ($\text{lag} > 0$) ограничено сверху величиной $\max(r_{\max}, q)$: ожидание возникает лишь пока обслуживается чужой запрос (не дольше q) либо пока виртуальное время не догонит ve собственного длинного запроса.

Следствие 2 (равномерный квант) [5]. Если все запросы клиента k не превышают кванта, $r^{(k)} \leq q$, то

$$-q < \text{lag}_k(t) < q. \quad (2.13)$$

Лемма 4 (оптимальность границ) [5]. Для любого пропорционально- долевого алгоритма с квантами размера q существуют конфигурации, в которых $\text{lag}_k(t)$ сколь угодно близок к $-q$, и конфигурации, в которых он сколь угодно близок к q .

Доказательство строится на примере n клиентов с равными весами: клиент, получивший первый квант, имеет $\text{lag} = q/n - q \rightarrow -q$ при $n \rightarrow \infty$, а клиент, получивший последний из первых n квантов, имеет $\text{lag} = q - q/n \rightarrow q$. Следовательно, границы $(-q, q)$ Следствия 2 неулучшаемы в классе пропорционально- долевого алгоритмов с квантом q , и EEVDF их достигает.

В стационарной системе с постоянным составом клиентов EEVDF эквивалентен алгоритму Earliest Deadline First (EDF). Фильтр доступности ($ve \leq V(t)$) отличает EEVDF от алгоритмов справедливой очередизации (Packet-by-Packet Fair Queueing), ограничивая отставание до $O(1)$ вместо $O(n)$ [5].

2.4 Аукционная модель A1349

В отличие от EEVDF, рассматривающего вычислительный ресурс как однородный, аукционная модель A1349 трактует выбор ядра как задачу распределения недолговечного ресурса между конкурирующими агентами. Формализация опирается на модель динамического механизма Ryuji Sano [18] для ресурсов с различной длительно-

стью использования и на результаты Vincent Leon и S. Rasoul Etesami [19] по онлайн-обучению в динамических VCG-механизмах.

Платформа и агенты.

Гетерогенная платформа описывается двумя множествами ядер,

$$\mathcal{P} = \{P_1, \dots, P_{K_P}\}, \quad \mathcal{E} = \{E_1, \dots, E_{K_E}\}, \quad (2.14)$$

где

- $\mathcal{P}$ — ядра высокой производительности (P-cores),
- $\mathcal{E}$ — энергоэффективные ядра (E-cores).

Время разделено на дискретные кванты $t = 1, 2, \dots$. В каждый момент времени ядро представляет собой *недолговечный* (perishable) ресурс (неиспользованный квант теряется безвозвратно).

Каждая задача $i \in \mathcal{N}^t$ характеризуется типом

$$\theta_i = (v_i, l_i), \quad (2.15)$$

где

- $v_i \in [0, \bar{V}]$ — ценность завершения задачи для пользователя,
- $l_i \in \{1, \dots, L\}$ — требуемое число квантов на P-ядре.

Тип θ_i задаётся внешне (см. Допущение 10 раздела 2.2) и не сообщается задачей через канал механизма. Терминология «агент», «тип», «аукцион» используется в формальном смысле теории проектирования механизмов: она задаёт правило аллокации и формулу платежа, но не требует стратегического поведения со стороны задач. Поскольку P/E-ядра разделяют общий набор инструкций, задача может быть назначена на любой класс ядер (выбор типа является решением механизма, а не задачи). На E-ядре длина задачи увеличивается в $\sigma > 1$ раз и составляет $\lceil l_i \cdot \sigma \rceil$ квантов, где $\sigma = \eta_P / \eta_E$ выражает отношение производительностей. Задача i получает полезность v_i только при исполнении полного контракта длиной $m_\kappa(l_i)$ квантов.

Каждой задаче назначен начальный бюджет $B_i > 0$, определяемый её классом обслуживания (real-time, interactive, batch, background). Остаточный бюджет в момент t обозначается

$$B_i^t = B_i - \sum_{s < t} p_i^s. \quad (2.16)$$

Аллокация задаче i в момент t допустима, если VCG-платёж не превышает остаточного бюджета, $p_i^t \leq B_i^t$. После победы на аукционе платёж амортизируется по квантам начатого контракта (см. ниже), что обеспечивает $B_i^{t+m} \geq 0$ к моменту его завершения. Принадлежность к классу χ_i является внешним параметром и не входит в тип θ_i .

MDP-формулировка.

Процесс планирования формализуется как марковский процесс принятия решений (MDP). Состояние в момент t записывается в виде

$$s^t = (\mathbf{x}^t, \mathbf{b}^t), \quad (2.17)$$

где

- $\mathbf{x}^t \in \{0, \dots, \lceil L\sigma \rceil - 1\}^K$ — остаточные длительности контрактов на каждом из $K = K_P + K_E$ ядер (верхняя граница учитывает максимальную длину контракта на Е-ядре),
- $\mathbf{b}^t$ — остаточные бюджеты активных задач.

Действие a^t задаёт для каждой задачи распределение по типам ядер,

$$a_i^t = (a_{i,P}^t, a_{i,E}^t) \in \{0, 1\}^2, \quad a_{i,P}^t + a_{i,E}^t \leq 1, \quad (2.18)$$

с ограничениями на число свободных ядер каждого типа K_P^t, K_E^t ,

$$\sum_{i \in \mathcal{N}^t} a_{i,P}^t \leq K_P^t, \quad \sum_{i \in \mathcal{N}^t} a_{i,E}^t \leq K_E^t. \quad (2.19)$$

Следующее состояние определяется функцией перехода $s^{t+1} = G(s^t, a^t, \mathcal{N}^{t+1})$, где $\mathcal{N}^{t+1}$ — множество вновь прибывших задач. Длительность контракта на каждом ядре обновляется по правилу

$$x_k^{t+1} = \begin{cases} m_\kappa(l_i) - 1, & \text{если ядро } k \text{ назначено задаче } i, \\ \max(x_k^t - 1, 0), & \text{иначе.} \end{cases} \quad (2.20)$$

Остаточные бюджеты уменьшаются на величину платежа,

$$b_i^{t+1} = b_i^t - p_i^t. \quad (2.21)$$

Контракты и стоимость.

Базовая стоимость одного кванта на ядре типа $\kappa \in \{P, E\}$ обозначается c_P, c_E , причём $c_P > c_E$ (отношение $\gamma = c_P/c_E$ введено в таблице обозначений). Полная стоимость контракта для задачи i :

$$\text{cost}(i, P) = c_P \cdot l_i, \quad \text{cost}(i, E) = c_E \cdot \lceil l_i \cdot \sigma \rceil. \quad (2.22)$$

Из-за округления вверх отношение $\lceil l_i \sigma \rceil / l_i$ зависит от l_i и не равно константе σ . Е-ядро оказывается дешевле Р-ядра, если экономия на стоимости кванта перевешивает удлинение контракта,

$$\frac{\lceil l_i \sigma \rceil}{l_i} < \gamma, \quad (2.23)$$

и дороже в обратном случае. Поэтому выбор класса ядра зависит от длины задачи l_i и не сводится к сравнению σ с γ .

Эффективная ценность задачи на каждом типе ядра с учётом стоимости квантов:

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot \lceil l_i \cdot \sigma \rceil. \quad (2.24)$$

Расширенная социальная функция благосостояния удовлетворяет уравнению Беллмана:

$$W(s^t) = \max_{a^t} \left[\sum_{i \in \mathcal{N}^t} (a_{i,P}^t \phi_P(\theta_i) + a_{i,E}^t \phi_E(\theta_i)) + \delta \cdot \mathbb{E}[W(s^{t+1})] \right], \quad (2.25)$$

где $\delta \in (0, 1)$, коэффициент дисконтирования. В настоящей работе используется дисконтированная формулировка, следуя Ruci Sano [18]. Результаты Vincent Leon и S. Rasoul Etesami [19] по онлайн-обучению доказаны для average-reward MDP в предположении неприводимости со спектральным зазором α . Перенос гарантий сожаления (regret) на дисконтированную постановку требует учёта эффективного горизонта $1/(1 - \delta)$ и контроля ошибки приближения bias-функции, что выходит за рамки настоящей работы. Приводимые ниже оценки сожаления используются как ориентир порядка убывания при $T \rightarrow \infty$, а не как точная гарантия для дисконтированной модели A1349. Платёж в виде (2.27), содержащий $\delta^{m_\kappa(l)}$, имеет смысл только при $\delta < 1$. Для average-reward формулировки требуется переписать платёж через разности относительных функций ценности.

Динамический VCG-механизм.

Обозначим $m_\kappa(l)$ длину контракта задачи на ядре типа κ : $m_P(l) = l$, $m_E(l) = \lceil l \cdot \sigma \rceil$.

Платёж задачи i , получившей ядро типа κ в момент t , определяется по правилу VCG (Викри–Кларка–Гровса) как разность между социальным благом без её участия и вкладом в текущее распределение:

$$p_i^t = W_{-i}(s^t) - W(s^t) + \phi_\kappa(\theta_i). \quad (2.26)$$

Здесь $W(s^t)$ — суммарное социальное благосостояние, включающее реализованный вклад $\phi_\kappa(\theta_i)$ победителя i , а $W_{-i}(s^t)$ обозначает оптимальное значение функции Беллмана той же MDP, в которой оптимизация проводится по всем задачам, кроме i . Формула эквивалентна стандартной маргинальной разности $W_{-i}(s^t) - [W(s^t) - \phi_\kappa(\theta_i)]$: благосостояние без участия i минус благосостояние с исключённым вкладом i . Интуитивно задача оплачивает экстерналию, налагаемую ею на остальные активные задачи. В Беллмановой формулировке полная стоимость контракта учитывается однократно в момент аллокации. Для согласования с бюджетным ограничением $\sum_s p_i^s \leq B_i$ единый платёж p_i^t разносится равными долями $p_i^s = p_i^t / m_\kappa(l_i)$ по квантам контракта. Это разнесение является номинальным и сохраняет $\sum_s p_i^s = p_i^t$, но не дисконтированную

сумму $\sum_s \delta^{s-t} p_i^s$. Строгая эквивалентность с правилом индифферентности контрактов Sano [18] (два контракта с одинаковой дисконтированной суммой $\sum_s \delta^{s-t} p_i^s$ неразличимы для агента) требует дисконтированно взвешенного распределения долей. В настоящей работе используется равномерное разнесение как приближение, оправданное малыми $1 - \delta$ на временных масштабах одного контракта.

В частном случае одного свободного ядра типа κ и двух претендующих задач i^* (победитель) и j (вторая по эффективной ценности) расчёт VCG-платежа принимает форму, аналогичную однородной модели Sano [18]. Без агента i^* ядро было бы выделено j на $m_\kappa(l_j)$ квантов, после чего ожидаемое будущее благосостояние составит $\delta^{m_\kappa(l_j)} \bar{W}_\kappa$, где $\bar{W}_\kappa$ — ожидаемое значение Беллмана при возвращении ядра типа κ в свободное состояние. В присутствии i^* ядро занимает на $m_\kappa(l_{i^*})$ квантов, что даёт $\delta^{m_\kappa(l_{i^*})} \bar{W}_\kappa$. Заменяя оценку V_j из однородной модели на эффективную стоимость $\phi_\kappa(\theta_j) = v_j - c_\kappa m_\kappa(l_j)$ (линейная стоимость ресурса уже вычтена в социальной функции благосостояния), получаем

$$p_{i^*} = \phi_\kappa(\theta_j) + (\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa. \quad (2.27)$$

Формула адаптирует результат [18] на гетерогенную платформу с отдельным учётом будущего благосостояния по типу ядра.

Онлайн-обучение механизма.

При неизвестных распределениях типа задач и переходного ядра MDP механизм обучается онлайн. Горизонт времени T разбивается на эпизоды $k = 1, 2, \dots$ возрастающей длительности. Каждый эпизод k состоит из двух фаз: фазы перемешивания (mixing) длительностью (Лемма 2 у Leon, Etesami [19])

$$d^{[k]} = \frac{\log k}{\alpha |\mathcal{S}_{\text{MDP}}|}, \quad (2.28)$$

в течение которой текущая политика $\pi^{[k]}$ применяется без сбора платежей для приближения марковской цепи к стационарному распределению, и стационарной фазы длительностью (Лемма 3 у Leon, Etesami [19])

$$l^{[k]} = \frac{\max\{4, \sqrt{k}\} \log(|\mathcal{A}_{\text{MDP}}| |\mathcal{S}_{\text{MDP}}| k / \zeta)}{\alpha \xi}, \quad (2.29)$$

в которой та же политика $\pi^{[k]}$ применяется с одновременным сбором платежей и оценок вознаграждений. Параметры:

- $\mathcal{S}_{\text{MDP}}, \mathcal{A}_{\text{MDP}}$ — пространства состояний и действий MDP (отличать от S_i обслуживания и $\mathcal{A}(t)$ активных клиентов из раздела 2.3),
- α — нижняя граница спектрального зазора цепи переходов,

- ξ — параметр сжатия политопа вероятностей (Определение 1 у Leon, Etesami [19], не путать с коэффициентом дисконтирования δ),
- ϵ — слабина правдивости и эффективности (truthfulness/efficiency slack) у Leon, Etesami [19],
- $\zeta \in (0, 1)$ — параметр доверия для UCB/LCB-оценок.

По завершении эпизода оценки $\hat{R}, \hat{P}$ строятся по принципу верхней доверительной границы (UCB/LCB) и обновляют политику решением $(n+1)$ линейных программ. Vincent Leon и S. Rasoul Etesami [19] доказывают в Теореме 2 оценки сожаления (regret) для социального благосостояния, платежей и стоимости предложений:

$$\begin{aligned} \text{Reg-SW} &= \tilde{O}(\epsilon T n + \alpha^{-4/3} \xi^{-1/3} |\mathcal{S}_{\text{MDP}}|^{-1/2} T^{2/3} n), \\ \text{Reg-SELL, Reg-BID} &= \tilde{O}(\epsilon T n^2 + \alpha^{-4/3} \xi^{-1/3} |\mathcal{S}_{\text{MDP}}|^{-1/2} T^{2/3} n^2), \end{aligned}$$

где n — число агентов. При известном горизонте T выбор $\epsilon \leq O(T^{-1/3})$ при ξ , согласованном по Теореме 2 и Замечанию 2 у Leon, Etesami [19], обеспечивает сублинейный рост $\tilde{O}(T^{2/3} n^2)$ для платежей и предложений (и $\tilde{O}(T^{2/3} n)$ для социального благосостояния), совпадающий с нижней границей $\Omega(T^{2/3})$ для задачи многоруких бандитов и эпизодических MDP с точностью до логарифмического множителя.

Свойства механизма.

В силу нестратегичности задач (см. Допущение 10) требования совместимости стимулов и индивидуальной рациональности не входят в число операционных целей A1349. VCG-аппарат заимствуется ради правила распределения, максимизирующего социальное благосостояние, и формулы платежей как маргинального вклада, естественно лежащей на роль единицы бюджета. Приводимые ниже свойства наследуются из работ Sano [18] и Leon, Etesami [19] и сохраняются в той мере, в какой соответствующие условия выполняются для модели A1349.

Теорема 2 (байесовская совместимость стимулов, Ryuji Sano [18], Теорема 1).

Динамический прямой механизм является байесовски совместимым по стимулам тогда и только тогда, когда для каждой задачи i : (1) вероятность получения ядра $\alpha_i(v_i, l_i)$ не убывает по v_i при фиксированном l_i ; (2) $\int_0^{v_i} \alpha_i(\nu, l_i) d\nu$ не возрастает по l_i при фиксированном v_i ; (3) существует $\underline{\Pi}_i$, не зависящее от l_i , такое что $\Pi_i(v_i, l_i) = \underline{\Pi}_i + \int_0^{v_i} \alpha_i(\nu, l_i) d\nu$.

Усиление до доминирующих стратегий обеспечивается динамическим VCG-механизмом с детерминированным монотонным распределением (Sano [18], Предложение 1 совместно с Теоремой 2). Если бы тип (v_i, l_i) сообщался задачей, правдивое сообщение было бы доминирующей стратегией. В A1349 тип задаётся внешним образом, поэтому DSIC-свойство наследуется как формальная характеристика VCG-аппарата, но операционно не используется. Класс обслуживания χ_i аналогично задаётся вне канала сообщений механизма.

Следствие 1 (индивидуальная рациональность, Ryūji Sano [18], Теорема 2).

При оптимальной политике π^* и соответствующих VCG-платежах индивидуальная полезность задачи неотрицательна, $u_i(\pi^*, p^*) \geq 0$. По стандартному свойству VCG она равна маргинальному вкладу задачи в социальное благосостояние, $u_i(\pi^*, p^*) = W(\pi^*) - W_{-i}(\pi_{-i}^*)$, где π_{-i}^* — оптимальная политика в задаче без агента i .

Предложение 1 (бюджетная допустимость).

Если VCG-платёж превышает остаточный бюджет, $p_i^*(s, a) > B_i^t$, задача не получает ядро в данном кванте. Модифицированное распределение $\tilde{a}^t = \arg \max_a \sum_{i: p_i \leq B_i^t} [a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)] + \delta \mathbb{E}[W(s^{t+1})]$ максимизирует социальное благосостояние на множестве бюджетно-допустимых аллокаций.

Жёсткое бюджетное ограничение в общем случае несовместимо с правилом VCG-платежей: исключение задачи при $p_i^* > B_i^t$ нарушает монотонность распределения по v_i и, как следствие, выводит механизм за рамки условий совместимости стимулов из Теоремы 2 (Dobzinski, Lavi, Nisan [20] о бюджетных аукционах). В рамках сделанного выше допущения о нестратегичности задач это не является препятствием, и Предложение 1 формулируется как свойство аллокации, а не как утверждение о совместимости стимулов (IC).

Таким образом, модель A1349 предлагает альтернативу пропорционально-долевому планированию — аукционный механизм, учитывающий гетерогенность ядер, бюджетные ограничения задач и динамику системы во времени.

3 Реализация алгоритмов планирования

3.1 Реализация EEVDF с помощью sched_ext

Данный раздел описывает реализацию математической модели EEVDF (раздел 2.3) в виде планировщика для платформы sched_ext. Представленная здесь реализация имеет вспомогательный характер и приводится для демонстрации общего подхода к отображению математических моделей планировщиков на платформу sched_ext.

Переменные состояния. Глобальное состояние системы хранится в BPF_MAP-структуре типа BPF_MAP_TYPE_ARRAY с единственным элементом структуры eevdf_ctx. Структура содержит два поля:

- `vtime_now` — системное виртуальное время $V(t)$ (уравнение (2.4));
- `total_weight` — суммарный вес активных задач $\sum_{j \in \mathcal{A}(t)} w_j$ (знаменатель $V(t)$).

Виртуальное время хранится в формате с фиксированной точкой и масштабируется коэффициентом `SCALE` = 1000. Среда eBPF не поддерживает арифметику с плавающей точкой, а приращения $V(t)$ при умеренной нагрузке имеют величину порядка единицы и обнулились бы при прямом целочисленном делении на $\sum w_j$. Масштабирование на `SCALE` сохраняет три младших разряда дроби и устраняет потерю точности. Операции сравнения $ve \leq V(t)$ при этом остаются корректными, так как обе величины хранятся в одинаковых единицах.

Состояние отдельной задачи хранится в BPF_MAP-структуре типа BPF_MAP_TYPE_TASK_STORAGE, что обеспечивает обращение за $O(1)$ без хеш-поиска и автоматическое освобождение при завершении задачи. Структура `task_ctx` содержит виртуальное время доступности `ve` (соответствует $ve^{(k)}$ в уравнении (2.6)), виртуальный дедлайн `vd` ($vd^{(k)}$, уравнение (2.7)) и флаг нахождения в очереди `on_rq`. Поля `ve` и `vd`, как и глобальное `vtime_now`, хранятся в одних и тех же масштабированных единицах виртуального времени.

Единственная очередь диспетчеризации `SHARED_DSQ` создаётся при инициализации и упорядочена по виртуальному дедлайну `vd`. При вставке через `scx_bpf_dsq_insert_vtime` ядро использует поле `vtime` как ключ красно-чёрного дерева, что реализует правило выбора EEVDF — среди доступных запросов извлекается тот, чей `vd` минимален.

Поверх `SHARED_DSQ` фреймворк sched_ext предоставляет на каждое ядро встроенную локальную очередь `SCX_DSQ_LOCAL`, из которой планировщик ядра берёт следующую задачу для исполнения. Перенос задач из общей очереди в локальные выполняется в обработчике `dispatch` вызовом `scx_bpf_dsq_move_to_local`: при освобождении

ядра вызывается `dispatch` для этого ядра, который переносит голову `SHARED_DSQ` (задачу с минимальным vd) в его локальную очередь. На быстром пути пробуждения `select_cpu` задача может быть вставлена напрямую в `SCX_DSQ_LOCAL` свободного ядра, минуя `SHARED_DSQ`; в этом случае обработчик `enqueue` для данной задачи пропускается.

Отображение обработчиков на модель. В таблице 3.1 приведено соответствие между обратными вызовами `sched_ext` и шагами алгоритма EEVDF.

Таблица 3.1: Соответствие обработчиков `sched_ext` модели EEVDF

Обработчик	Реализуемый шаг модели
<code>select_cpu</code>	Быстрый путь пробуждения при $ve \leq V(t)$ на свободном ядре
<code>enqueue</code>	Ограничитель отставания (теорема 1) и вставка по $vd = ve + q/w_i$ (уравнение (2.7))
<code>dispatch</code>	Извлечение задачи с минимальным vd
<code>stopping</code>	Продвижение ve , vd и $V(t)$ на $\Delta V = u / \sum w_j$ (уравнение (2.4))
<code>enable</code>	Вход задачи: $ve := V(t)$, $\sum w_j += w_i$ (уравнение (2.10))
<code>disable</code>	Выход задачи: коррекция $V(t)$ по уравнению (2.9)
<code>set_weight</code>	Коррекция $V(t)$ по уравнению (2.11)
<code>running</code>	Пустой обработчик
<code>init_task</code>	Выделение <code>task_ctx</code>
<code>init</code>	Создание <code>SHARED_DSQ</code> , инициализация <code>eevdf_ctx</code>
<code>exit</code>	Передача кода завершения агенту (UEI)

Ключевые аспекты реализации.

Атомарное продвижение $V(t)$. Системное виртуальное время обновляется при каждом снятии задачи с процессора (обработчик `stopping`). Поскольку на многоядерной системе несколько ядер могут одновременно вызвать `stopping`, продвижение V выполняется через атомарную операцию `__sync_fetch_and_add`. Величина приращения

$$\Delta V = \frac{u \cdot \text{SCALE}}{\sum_j w_j}, \quad (3.1)$$

где u — фактически использованная часть кванта (`SCX_SLICE_DFL - p->scx.slice`), является дискретной формой непрерывного правила $dV/dt = 1 / \sum w_j$ из уравнения (2.4).

Двусторонний ограничитель отставания. Реализация фиксирует длину запроса равной размеру кванта, $r = q = \text{SCX_SLICE_DFL}$ (20 мс), что согласует дискретное продвижение времени с границами Теоремы 1. При постановке задачи в очередь (обработчик `enqueue`) применяется ограничение

$$V(t) - q_v \leq ve' \leq V(t) + q_v,$$

где $q_v = q \cdot \text{SCALE}/w_i$ — виртуальная стоимость одного кванта. В коде ограничение реализовано условиями $ve' < V(t) - q_v \Rightarrow ve' := V(t) - q_v$ и $ve' > V(t) + q_v \Rightarrow ve' := V(t) + q_v$, что эквивалентно приведённой формуле. Нижняя граница не позволяет задаче, накопившей значительный положительный lag за время сна, монополизировать ресурс по возвращении. Верхняя граница не позволяет задаче уйти в далёкое будущее виртуального времени.

Теорема 1 даёт асимметричные границы $-r_{\max} < \text{lag}_k(t) < \max(r_{\max}, q)$, тогда как реализованный ограничитель симметричен. При принятом условии $r = q$ обе границы совпадают по модулю с q , поэтому симметричное ограничение $|V - ve| \leq q_v$ покрывает их с запасом и не нарушает асимптотических оценок. Симметричная форма выбрана ради простоты eBPF-кода и постоянного числа ветвлений в обработке.

Обработка динамических событий. Реализация следует первой из трёх стратегий, описанных в разделе 2.3: отставание задачи сохраняется при выходе и переносится в момент возврата, а виртуальное время системы корректируется по формулам Леммы 2 для сохранения инварианта $\sum \text{lag}_i = 0$. При входе задачи (**enable**) её виртуальное время доступности инициализируется как $ve = V(t)$, что соответствует нулевому отставанию в момент входа. Суммарный вес обновляется атомарно. При выходе (**disable**) виртуальное время корректируется по уравнению (2.9): $V(t^+) = V(t) + \text{lag}/\sum w_{j \neq i}$. В реализации lag вычисляется в виртуальных единицах как $V(t) - ve$ и связан с теоретическим определением $\text{lag}_i = S_i - s_i$ множителем $1/w_i$. Знак при этом сохраняется: положительное значение означает недообслуженность как в модели, так и в реализации.

При изменении веса (**set_weight**) применяется двухшаговая коррекция по уравнению (2.11), эквивалентная последовательности выход–вход с новым весом.

Таким образом, реализация сохраняет все инварианты модели EEVDF при ограничениях среды eBPF: отсутствии блокирующих вызовов, конечных циклах и фиксированных структурах данных. Единственное отклонение от теоретической модели состоит в замене непрерывного продвижения $V(t)$ дискретными приращениями на границах квантов, что не нарушает границ теоремы 1, поскольку q конечно.

3.2 Реализация аукционной модели A1349

Данный раздел описывает реализацию аукционной модели A1349 (раздел 2.4) в виде eBPF-планировщика `scx_A1349`, построенного средствами `sched_ext`. Задачи во всех очередях упорядочены непосредственно по эффективной ценности $\phi_\kappa(\theta_i)$, а одноюнитный VCG-платёж по формуле (2.27) рассчитывается на этапе диспетчеризации путём чтения двух старших кандидатов из кластерной DSQ через итератор `bpf_iter_scx_dsq`.

Операционная редукция модели. Полный аппарат раздела 2.4 включает онлайн-обучение политики по схеме Leon-Etesami [19], гарантирующее совместимость стимулов и сублинейное сожаление при стратегических участниках. По Допущению 10 задачи нестратегичны, поэтому DSIC-свойство и точное встречное благосостояние $W_{-i}(s^t)$ операционно не нужны, и обучающий контур не реализуется. VCG-аппарат используется как правило максимизации общественного благосостояния: $\arg \max_\kappa \phi_\kappa$ максимизирует социальное благосостояние независимо от стратегичности, а платёж выражает упущенную ценность конкурента и служит единицей бюджетного регулирования. Реализация сохраняет правило $\kappa^* = \arg \max_\kappa \phi_\kappa$ и форму платежа (2.27), заменяя ожидание Беллмана $\bar{W}_\kappa$ кластерной EWMA-оценкой реализованных ϕ (обновляется в обработчике `stopping`). Все расчёты выполняются за $O(1)$ на событие и укладываются в ограничения верификатора eBPF (раздел 1.3) благодаря предвычисленной в пользовательском пространстве таблице δ^m .

Архитектура очередей диспетчеризации. При инициализации создаются три кластерные DSQ и набор привязанных к ядрам очередей:

- `AUCTION_DSQ_P` (`id = 1`) — очередь задач, назначенных на P-кластер (множество $\mathcal{P}$)
- `AUCTION_DSQ_E` (`id = 2`) — очередь задач, назначенных на E-кластер (множество $\mathcal{E}$)
- `AUCTION_DSQ_STARVED` (`id = 3`) — очередь задач с исчерпанным бюджетом (нарушение бюджетного ограничения B_i^t)
- `AUCTION_DSQ_PERCPU_BASE + CPU_NUM` (`id = 100 + cpu`) — по одной очереди на ядро для долго работающих задач, упорядоченная по ϕ . На этапе `init` создаётся `AUCTION_NCPU_MAX = 64` таких очередей. Они сохраняют кэш-локальность между квантами и образуют отдельный уровень иерархии диспетчеризации в отличие от `SCX_DSQ_LOCAL`, который служит безусловным FIFO-слотом ближайшего исполнения.

Все четыре типа DSQ упорядочены по приоритетному ключу $\text{encode}(\phi_{\kappa^*}(\theta_i))$, который

кодирует знаковую эффективную ценность в положительное число u64 как

$$\text{encode}(\phi) = \begin{cases} \text{PHI_BIAS} - \phi, & \phi \geq 0, \\ \text{PHI_BIAS} + |\phi|, & \phi < 0, \end{cases} \quad \text{PHI_BIAS} = 2^{62}.$$

Большее ϕ соответствует меньшему ключу и попадает в начало очереди, что согласуется со встроенным в `sched_ext` правилом сортировки DSQ по возрастанию `vtime`. Типичный диапазон $|\phi|$ ограничен весом задачи (после описанной ниже перенормировки $|\phi| \lesssim 5 \cdot 10^4$), что заведомо укладывается в 2^{62} .

Переменные состояния. Состояние планировщика разделено на три части, различающиеся по владельцу:

- конфигурационная BPF_MAP-структура `global_data` (тип `BPF_MAP_TYPE_ARRAY`, значение — `auction_ctx`) обновляется пользовательским агентом и используется BPF только для чтения
- исполнительная BPF_MAP-структура `runtime_data` (тип `BPF_MAP_TYPE_ARRAY`, значение — `auction_runtime`) хранит EWMA-оценки $\bar{W}_P$, $\bar{W}_E$ и обновляется только BPF
- по-задачное состояние `auction_task_ctx` в BPF_MAP-структуре типа `BPF_MAP_TYPE_TASK_STORAGE` с флагом `BPF_F_NO_PREALLOC`.

Разделение конфигурационной и исполнительной структур исключает гонки между периодическим обновлением параметров платформы пользовательским пространством и накоплением EWMA-оценок в BPF.

В таблице 3.2 приведены основные переменные состояния и их соответствие символам модели.

Таблица 3.2: Переменные состояния планировщика A1349

Переменная	Символ модели	Описание
max_capacity, min_capacity	η_P, η_E	Вычислительная мощность P- и E-ядер из <code>cpu_capacity</code>
cost_p, cost_e	c_P, c_E	Стоимости одного кванта на ядре каждого типа
p_core_count, e_core_count	K_P, K_E	Число P/E ядер на платформе
w_bar_p, w_bar_e	$\bar{W}_P, \bar{W}_E$	EWMA-оценка ожидаемого будущего благосостояния на каждом кластере
phi_enq	$\phi_{\kappa^*}(\theta_i)$	Эффективная ценность задачи, выбранная при постановке в очередь
m_enq	$m_{\kappa}(l_i)$	Длина контракта, используемая в индексе таблицы δ^m
budget	B_i^t	Остаточный бюджет задачи
budget_max	$B_i^{\max}$	Максимальный бюджет ($w_i \cdot \text{BUDGET_MUL}$)
len_est_ns	l_i (нс-прокси)	EWMA длительности исполнения, прокси для l_i
on_p_type	$\kappa(i_t)$	Тип кластера, на который задача назначена в текущем кванте
long_slice	—	Признак выдачи E-кванта (для учёта в <code>stopping</code>)
last_stop_ns	—	Момент последнего истинного засыпания, основа для пополнения бюджета
wake_prev_cpu	—	Подсказка кэш-привязки, сохранённая в <code>select_cpu</code>
delta_table[m]	δ^m	Предвычисленная в пользовательском пространстве таблица степеней дисконта (fixed-point, шкала 2^{20})
cpu_is_p[cpu]	—	Классификация ядер на P/E, заполняется пользовательским агентом

Отображение обработчиков на модель. Соответствие обработчиков шагам аукционного механизма приведено в таблице 3.3.

Таблица 3.3: Соответствие обработчиков sched_ext аукционной модели

Обработчик	Реализуемый шаг модели
select_cpu	P-bias сканирование простаивающих ядер (см. ниже). При найденном свободном ядре прямая вставка в SCX_DSQ_LOCAL с P-квантом (аукцион пропускается ввиду отсутствия конкуренции). Запоминается prev_cpu для кэш-привязки на медленном пути
enqueue	Вычисление ϕ_P , ϕ_E (уравнение (2.24)); выбор кластера κ^* при пробуждении (иначе сохранение ранее выбранного); длина контракта $m_\kappa(l_i)$; бюджетная проверка B_i^t (при $B_i^t < B_i^{\max}/10$ задача направляется в AUCTION_DSQ_STARVED); асимметричное P→E перенаправление коротких задач; привязанная к ядру очередь для длительных вытесненных задач; кэш-привязка к prev_cpu при пробуждении
dispatch	Четырёхуровневое извлечение: привязанная к ядру очередь, далее аукцион в локальном кластере, далее кросс-кластерное заимствование, далее STARVED (см. ниже)
running	Пустой обработчик
stopping	Учёт фактически потраченного времени кванта; обновление EWMA длительности len_est_ns ($\alpha = 1/8$); обновление EWMA $\bar{W}_\kappa$ ($\alpha = 1/16$); фиксация last_stop_ns только при истинном засыпании (runnable = false)
enable	Приём задачи в планировщик: $B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$, $B_i^0 = B_i^{\max}$ (полностью оплачена при входе), len_est_ns = SLICE_P
disable	Удаление состояния задачи через bpf_task_storage_delete
set_weight	Пересчёт $B_i^{\max}$ и пропорциональное масштабирование остаточного бюджета
init	Создание кластерных DSQ, очереди STARVED, набора AUCTION_NCPU_MAX = 64 привязанных к ядрам очередей; заполнение значений стоимостей по умолчанию

Вычисление эффективной ценности. Модель определяет эффективную ценность задачи на ядре типа κ как (уравнение (2.24)):

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot \lceil l_i \cdot \sigma \rceil.$$

В реализации v_i аппроксимируется весом задачи w_i (p->scx.weight), а длина l_i приводится к безразмерным P-квантам через нормировку наносекундной EWMA длительно-

сти:

$$l_q = \frac{\hat{l}_{\text{ns}}}{\text{SLICE_P}}.$$

Получаемые компоненты ϕ имеют тот же масштаб, что и веса задач:

$$\phi_P = w_i - c_P \cdot l_q, \quad \phi_E = w_i \cdot \frac{\eta_E}{\eta_P} - c_E \cdot l_q.$$

Нормировка делением на `SLICE_P` продиктована совместимостью шкал. Прямая формула $\phi \sim w \cdot l_{\text{ns}}$ давала бы $|\phi| \sim 10^{10}$ при одновременном масштабе бюджета $\sim w \cdot \text{BUDGET_MUL} \sim 10^6$, что приводило бы VCG-платежи либо к отсечению в нуле, либо к однократному опустошению корзины токенов в одном такте диспетчеризации. После перенормировки $|\phi| \lesssim 5 \cdot 10^4$ совпадает по порядку с диапазоном весов `p->scx.weight` $\in [1, 49152]$ и согласуется как со шкалой бюджета, так и со шкалой платежей. Дисконт ценности на Е-ядре множителем η_E/η_P отражает то, что Е-квант доставляет в σ^{-1} раз меньше полезной работы (эквивалентно теоретическому штрафу $c_E \cdot [l_i \sigma]$ с противоположной стороны равенства). Деление наносекундной стоимости на длину кванта `SLICE_P` выполняется с округлением (прибавлением `SLICE_P/2` до целочисленного деления), что исключает систематическое смещение для подквантовых задач.

На уровне диспетчеризации реализация использует две длительности гранения кванта: `AUCTION_SLICE_P` = 20 мс для Р-кластера и `AUCTION_SLICE_E` = $(3 \cdot \text{SLICE_P})/2$ = 30 мс для Е-кластера. Увеличенный квант на Е амортизирует постоянные накладные расходы переключения контекста, относительный вклад которых выше на медленном ядре, и снижает частоту перепланирования длительных вычислительных задач.

Длина контракта вычисляется как $l_p = \lceil \text{len_ns}/\text{SLICE_P} \rceil$, $l_p \geq 1$, с $m_P(l) = l_p$ и $m_E(l) = \lceil l_p \cdot \eta_P/\eta_E \rceil$ (округление вверх). Значение насыщается на `MAX_CONTRACT_LENGTH` – 1 = 31, что соответствует горизонту $\approx 0,64$ с при Р-кванте 20 мс ($32 \cdot 20$ мс) и согласуется с предположением модели о конечной верхней границе L .

Бюджетный механизм. Каждой задаче назначается бюджет в виде корзины токенов (token bucket), моделирующей ограничение B_i из раздела 2.4. Максимальный бюджет пропорционален весу: $B_i^{\text{max}} = w_i \cdot \text{BUDGET_MUL}$, где `BUDGET_MUL` = 2000. При входе задачи в планировщик (обработчик `enable`) бюджет инициализируется полным значением, $B_i^0 = B_i^{\text{max}}$ — задача принимается в систему «полностью оплаченной». Пополнение выполняется при пробуждении (флаг `SCX_ENQ_WAKEUP`) пропорционально длительности сна:

$$\Delta B_i = \min(\Delta t_{\text{idle}}, T_{\text{max}}) \cdot \frac{w_i}{\text{REPLENISH_DIV}},$$

где $T_{\text{max}} = 1$ с (`REPLENISH_IDLE_CAP`) и `REPLENISH_DIV` = $5 \cdot 10^6$. Бюджет отсекается сверху значением B_i^{max} . Если на этапе `enqueue` остаточный бюджет $B_i^t < B_i^{\text{max}}/10$ (в коде эквивалентно $10 \cdot B_i^t < B_i^{\text{max}}$), задача направляется напрямую в `AUCTION_DSQ_STARVED`, не тратя такт диспетчеризации на заведомо неоплатоспособный аукцион. Это реализует Предложение 2.4 в виде дешёвой консервативной проверки на входе. Окончательная

проверка $p_{i^*} \leq B_{i^*}^t$ выполняется на этапе диспетчеризации с учётом фактического платежа.

Точный VCG-платёж. Платёж рассчитывается по формуле (2.27) непосредственно, без приближения резервной ценой. При вызове обработчика `dispatch` из кластерной DSQ через `bpf_iter_scx_dsq` извлекаются две головные задачи: победитель i^* и претендент j (вторая по ϕ задача). Платёж в реализации представлен как

$$p_{i^*} = \phi_\kappa(\theta_j) + \frac{\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})}}{\text{DELTA_SCALE}} \cdot \bar{W}_\kappa,$$

где значения δ^m извлекаются из таблицы `delta_table` (fixed-point со шкалой `DELTA_SCALE = 2^{20}`), предвычисленной в пользовательском пространстве на основе настраиваемого параметра δ (по умолчанию $\delta = 0,98$, $L = 32$, горизонт $\sim 0,64$ с при Р-кванте 20 мс). Если очередь содержит единственного претендента, полагается $\phi_j = 0$, $m_j = 0$ (то есть $\delta^0 = 1$), что даёт вырожденную форму

$$p_{i^*} = (1 - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa,$$

интерпретируемую как «оплата одинокого победителя» упущенной ценностью свободной траектории ядра. Отрицательный платёж отсекается в нуле: VCG-механизм никогда не пополняет бюджет.

Если $p_{i^*} \leq B_{i^*}^t$, бюджет уменьшается на p_{i^*} , и задача i^* переносится в локальную очередь ядра `SCX_DSQ_LOCAL`. В противном случае задача i^* перемещается в `AUCTION_DSQ_STARVED` с сохранением её ϕ -кодированного ключа (через `scx_bpf_dsq_move_set_vtime` перед `scx_bpf_dsq_move_vtime`, чтобы не нарушить дисциплину сортировки DSQ), а аукцион повторяется для нового верхнего элемента той же очереди. Число попыток в пределах одного вызова `dispatch` ограничено константой `DISPATCH_AUCTION_TRIES = 3`, что согласуется с требованием верификатора eBPF к ограниченному циклам. Если победитель не может исполняться на текущем ядре (нарушение `p->cpu_ptr`, типичный случай — `per-CPU kworker`'ы), он перенаправляется в `SCX_DSQ_LOCAL_ON` разрешённого простаивающего ядра (через `scx_bpf_pick_idle_cpu`, при отсутствии — `scx_bpf_pick_any_cpu`) и раунд считается несостоявшимся, чтобы цикл продолжил работу со следующего верхнего элемента.

Обучение $\bar{W}_\kappa$. Оценка $\bar{W}_\kappa$ обновляется в обработчике `stopping` как экспоненциально взвешенное скользящее среднее реализованной эффективной ценности задачи:

$$W_{\text{real}} = |\phi_\kappa(\theta_i)| \cdot \frac{t_{\text{consumed}}}{\text{SLICE_P}}, \quad \bar{W}_\kappa \leftarrow \frac{(\text{W_BAR_EWMA_DEN} - 1) \cdot \bar{W}_\kappa + W_{\text{real}}}{\text{W_BAR_EWMA_DEN}},$$

где `W_BAR_EWMA_DEN = 16`. Модуль ϕ обеспечивает $\bar{W}_\kappa \geq 0$ и интерпретируется как ожидаемое будущее благосостояние, служащее множителем для δ -разности при расчёте платежа. Нормировка на `SLICE_P` устраняет смещение между квантом Р-цикла (20 мс) и удлинённым квантом Е-цикла (30 мс): фактически потраченное время кванта приво-

дится к единой шкале «Р-квант». Длительность исполнения l_i обновляется отдельной EWMA с $\alpha = 1/8$: $\hat{l} \leftarrow (7\hat{l} + t_{\text{consumed}})/8$, более медленной для подавления шумовых колебаний ϕ между соседними квантами.

Маршрутизация и стационарность кластера. Решение аукциона $\kappa^* = \arg \max_{\kappa} \phi_{\kappa}$ пересматривается только при пробуждении задачи (флаг `SCX_ENQ_WAKEUP`) либо при её первом появлении в системе (`last_stop_ns = 0`). Вытеснение по истечении кванта (`preempt re-enqueue`) сохраняет ранее выбранный кластер `tctx->on_p_type`: повторная оценка ϕ между двумя соседними квантами одной задачи внесла бы шум в маршрутизацию без новой информации, а миграция в середине пробега обнуляет кэш и ухудшает пропускную способность памяти.

Кэш-привязка к `prev_cpu` при пробуждении реализуется так: если ядро, на котором задача исполнялась в последний раз, принадлежит выбранному кластеру, входит в маску `p->cpus_ptr` и в данный момент бездействует, задача вставляется напрямую в `SCX_DSQ_LOCAL_ON | prev_cpu`, минуя кластерный аукцион. Если текущий процессор не совпадает с `prev_cpu`, дополнительно выполняется `scx_bpf_kick_cpu` с флагом `SCX_KICK_IDLE`, чтобы целевое ядро немедленно вышло из простоя. Согласованность с моделью сохраняется: пустое ядро в кластере не создаёт конкуренции, и $\arg \max \phi$ принимает все доступные задачи.

Долго работающие задачи (`len_est_ns ≥ SLICE_P`), вытесненные на границе кванта (когда обработчик `enqueue` вызывается без флага `SCX_ENQ_WAKEUP` при ненулевом `last_stop_ns`), направляются не в кластерную DSQ, а в привязанную к текущему ядру очередь `AUCTION_DSQ_PERCPU_BASE + CPU_NUM`. Это предотвращает рассеяние CPU-связанных задач по ядрам кластера между квантами и сохраняет кэш-локальность, что критично для пропускной способности и латентности памяти.

Асимметричное перенаправление. Чистый $\arg \max \phi_{\kappa}$ не учитывает числа ядер в кластерах: на платформе с $K_P = 8$, $K_E = 16$ строгое предпочтение Р-кластера оставляло бы Е-ядра простаивать. Реализация добавляет перенаправление (`spill`) на основе нормализованной глубины очереди:

$$Q_P \geq K_P \quad \text{и} \quad Q_P \cdot K_E > Q_E \cdot K_P \quad \implies \quad \text{маршрутизация в Е.}$$

Перекрёстное умножение выбрано вместо деления, чтобы избежать 64-битной операции деления на быстром пути eBPF. Перенаправление активируется только для коротких задач ($\hat{l}_{\text{ns}} < \text{SLICE}_P$). Длительные memory-bound задачи остаются в выбранном по ϕ кластере, так как их перенос на Е ведёт к двукратному росту латентности и $\sim 30\%$ потере пропускной способности памяти (эмпирически измерено на PassMark MEM/MEM_LAT). При перенаправлении пересчитываются ϕ_E и $m_E(l_i)$, чтобы DSQ-ключ соответствовал реальному кластеру назначения.

Выбор ядра при пробуждении. Обработчик `select_cpu` реализует специализированное сканирование, отличающееся от вспомогательного `scx_bpf_select_cpu_df1`. Поведение по умолчанию имеет систематический уклон к `prev_cpu` и часто

оставляет однопоточные тесты на Е-ядре, в то время как для задач со среднестатистическим весом выполняется $\phi_P \geq \phi_E$. Реализация выполняет следующий порядок поиска:

1. быстрый путь: `prev_cpu` выбирается, если оно является Р-ядром, входит в `p->cpus_ptr` и в данный момент простаивает;
2. полный путь: `bpf_for`-цикл по диапазону $[0, 64)$ (константа `AUCTION_NCPU_MAX`) ищет любое простаивающее Р-ядро в маске задачи;
3. запасной путь: при отсутствии свободных Р-ядер вызывается `scx_bpf_select_cpu_dfl`, который при необходимости возвращает простаивающее Е-ядро.

Если найдено свободное ядро, задача вставляется напрямую в `SCX_DSQ_LOCAL` с Р-квантом и обработчик `enqueue` для этой задачи пропускается — образуется быстрый путь пробуждения. Поле `tctx->wake_prev_cpu` заполняется до сканирования и используется обработчиком `enqueue` для возможного кэш-привязочного `pin`'а на медленном пути, когда `select_cpu` не смог разместить задачу на свободном ядре.

Диспетчеризация и заимствование. Обработчик `dispatch` реализует четырёхуровневый приоритет извлечения:

1. привязанная к ядру очередь `AUCTION_DSQ_PERCPU_BASE + CPU_NUM` — наиболее кэш-тёплые задачи, обслуживаются первой через `scx_bpf_dsq_move_to_local`;
2. локальная кластерная очередь `AUCTION_DSQ_k` с полноценным аукционом по формуле (2.27), бюджетной проверкой и возможным перемещением неоплатоспособной задачи в `STARVED`. Если очередь содержит ровно одну задачу (`scx_bpf_dsq_nr_queued = 1`), аукцион вырождается в копию (нет претендента j), и реализация переходит на ускоренный путь `scx_bpf_dsq_move_to_local`, экономя выделение и освобождение итератора `DSQ`;
3. кросс-кластерное заимствование из `AUCTION_DSQ_k` прямым перемещением в локальную очередь ядра без повторного аукциона: нахождение задачи в чужом кластере означает, что её ϕ был рассчитан под этот кластер, а $\bar{W}_k$ заимствующего ядра внесла бы шум в платёж; сохранение работы (*work conservation*) важнее точности `VCG` в этой фазе;
4. очередь `STARVED` — последний приоритет, задачи с исчерпанным бюджетом обслуживаются ядрами обоих кластеров после пополнения через простой.

Дополнительно после успешной постановки задачи в кластерную очередь обработчик `enqueue` выполняет *work-conservation-будильник*: вызовом `scx_bpf_pick_idle_cpu` ищется любое свободное ядро из маски задачи и при его обнаружении (не совпадающем с текущим) выполняется `scx_bpf_kick_cpu` с флагом `SCX_KICK_IDLE`, чтобы вновь добавленная задача не ждала истечения кванта на соседнем ядре.

Пространство пользователя. Программа `sx_A1349.c` выполняет функции агента конфигурации. При запуске она:

- считывает мощности ядер из `/sys/devices/system/cpu/cpu*/cpu_capacity`, определяет $\eta_P = \max_{cpu} cap$, $\eta_E = \min_{cpu} cap$ и вычисляет $\sigma = \eta_P/\eta_E$
- классифицирует ядра по порогу `P_CAP_PCT = 90 %`: ядро считается Р-ядром, если его `cpu_capacity` $\geq 0,9 \cdot \eta_P$, иначе Е-ядром
- при отсутствии явного указания `-e` автоматически калибрует $c_E = c_P \cdot \eta_E/\eta_P$, чтобы выполнялось $\gamma = c_P/c_E = \sigma$
- вычисляет таблицу степеней дисконта $\delta^m \cdot 2^{20}$ для $m \in [0, \text{MAX_CONTRACT_LENGTH})$ в двойной точности и записывает её как `fixed-point` в `BPF_MAP`-структуру `delta_table`. Это исключает арифметику с плавающей точкой на BPF-стороне, не поддерживаемую верификатором.

Значения по умолчанию: $c_P = 1024$, $\delta = 0,98$. При запуске агент проверяет $c_P > 0$, $0 < \delta < 1$ и при пользовательском задании c_E — условие $c_P > c_E > 0$ (Р-ядро должно быть строго дороже Е-ядра). В режиме работы агент один раз в 5 секунд повторяет процедуру `refresh_cpu_capacities` для обнаружения `hotplug`-событий или динамического изменения `cpu_capacity` и обновляет `global_data` при изменении любого параметра.

Таким образом, реализация строит порядок исполнения непосредственно по эффективной ценности ϕ . Маргинальный VCG-платёж рассчитывается точно из одноюнитной формулы (2.27) с заменой неизвестного W_{-i} на EWMA-оценку $\bar{W}_\kappa$ реализованных ценностей. Дополнительные механизмы — Р-bias выбор ядра при пробуждении, кэш-привязка к `prev_cpu` с пробуждением целевого ядра, привязанные к ядрам очереди для длительных задач, асимметричное перенаправление коротких задач из перегруженного Р-кластера и ускоренный путь для одноэлементной очереди — обеспечивают практическую конкурентоспособность с EEVDF без отступления от формальной аукционной модели.

4 Описание тестового окружения и экспериментального оборудования

4.1 Тестовое оборудование и программное окружение

Экспериментальная оценка планировщика проведена на настольной платформе с гетерогенным процессором Intel Core Ultra 7 265K (кодовое название Arrow Lake-S). Характеристики платформы сведены в таблицу 4.1.

Таблица 4.1: Характеристики тестовой платформы

Параметр	Значение
Процессор	Intel Core Ultra 7 265K (Arrow Lake-S)
Р-ядра	8 × Lion Cove, до 5,5 ГГц
Е-ядра	12 × Skymont, до 4,6 ГГц
Потоков на ядро	1 (без Hyper-Threading)
Всего потоков	20
Кэш L2	3 МБ × 8 Р-ядер + 4 МБ × 3 модуля Е-ядер
Кэш L3 (общий)	30 МБ
Микрокод	0x121
Оперативная память	32 ГБ DDR5, 6000 МГц
TDP (PL1 / PL2)	125 Вт / 250 Вт
C-states	включены
Turbo Boost	включён
Turbo Boost Max 3.0	включён
Регулятор частоты	powersave (intel_pstate)
Операционная система	Arch Linux
Ядро	Linux 7.0
libbpf / sched_ext	в составе Linux 7.0
hackbench	2.10 (rt-tests)
sysbench	1.0.20
schbench	commit 6300b8f
PostgreSQL	18.3

Выбор платформы обусловлен прямым соответствием между аппаратной топологией и гетерогенной постановкой задачи. Процессор содержит два кластера ядер с различной производительностью. Отсутствие Hyper-Threading исключает влияние разделяемых ресурсов между аппаратными потоками внутри одного ядра и упрощает интерпретацию результатов, поскольку каждая задача получает полный набор исполнительных блоков ядра без конкуренции на уровне SMT.

В качестве операционной системы используется Arch Linux с ядром Linux версии 7.0 и включённой поддержкой sched_ext. Базовым планировщиком fair-класса служит EEVDF, который в экспериментах выступает контрольной линией. Предложенные

планировщики `scx_EEVDF` и `scx_A1349` подключаются к `sched_ext`. Регулятор частоты оставлен в значении по умолчанию. Управление тактовой частотой выполняется аппаратно (Hardware-managed P-states, HWP), а драйвер `intel_pstate` передаёт процессору подсказки энергоэффективности (EPP).

4.2 Набор бенчмарков

Для оценки планировщиков используется набор из трёх бенчмарков, каждый из которых генерирует характерный профиль нагрузки и измеряет отдельный аспект поведения системы. Бенчмарки запускаются последовательно в порядке `hackbench`, `sysbench`, `schbench` с интервалами охлаждения между фазами, чтобы каждый бенчмарк стартовал из одинакового исходного состояния процессора (температура и тактовые частоты) и последовательность тестов не влияла на результаты. Порядок планировщиков рандомизируется между прогонами, что усредняет вклад фоновых процессов между планировщиками. Близкие работы по `sched_ext` планировщикам опираются на схожий инструментарий. Например, Tang и соавторы [12] применяют `hackbench` при оценке SRAND, планировщика на основе глобальной очереди готовых задач.

hackbench. Утилита из пакета `rt-tests`. Создаёт группы процессов, обменивающихся сообщениями через каналы или сокет. Массовое порождение короткоживущих задач и интенсивное межпроцессное взаимодействие нагружает механизмы пробуждения, миграции и балансировки.

sysbench. Многопоточный бенчмарк с модулем `oltp_read_only`. Выполняет транзакции чтения к базе данных PostgreSQL. Поток многократно блокируется на ожидании ответа базы и просыпается лишь на время обработки ответа, что нагружает механизм пробуждения.

schbench. Бенчмарк задержек планирования, моделирующий веб-нагрузку. Каждый запрос рабочего потока состоит из двух фаз сна (имитация сети и диска) и фазы матричных вычислений. Поток-координатор ставит задания в очередь и ожидает результатов. Бенчмарк проверяет три аспекта планировщика — способность загрузить все ядра, поддержание длительных квантов без вытеснения и минимизацию задержки пробуждения. Бенчмарк наиболее чувствителен к качеству решений о размещении задач на ядрах, поскольку вытеснение потока во время матричных вычислений приводит к удержанию им блокировки в неисполняемом состоянии, что блокирует прогресс ожидающих потоков на других ядрах и снижает совокупную пропускную способность.

В дополнение к метрикам бенчмарков на протяжении всего эксперимента в фоновом режиме выполняется инструмент `sched_latency` (eBPF-программа, снимающая задержки планирования через точки трассировки ядра), а также считываются системные счётчики и показания подсистемы Intel RAPL. Полный перечень измеряемых величин с указанием единиц измерения и целевого направления оптимизации приведён в

разделе 4.3.

Каждый эксперимент проводится при трёх уровнях нагрузки — **лёгкая**, **умеренная** и **стрессовая** — различающихся параметрами запуска бенчмарков (таблица 4.2). На каждом уровне исполняются $N = 6$ прогонов каждого из четырёх планировщиков, всего 24 прогона на уровень и 72 прогона в эксперименте. Порядок планировщиков на каждой итерации выбирается случайной перестановкой из четырёх элементов. Длительность одного прогона определяется параметрами бенчмарка (sysbench и schbench запускаются на 30 секунд, hackbench — до завершения заданного числа сообщений), перед каждым прогоном выдерживается пауза 30 секунд для сброса частот процессора, при старте каждого бенчмарка дополнительно отбрасываются первые 5 секунд измерений в качестве разогрева. По результатам прогонов вычисляются выборочные средние, выборочные стандартные отклонения и 95%-доверительные интервалы по t -распределению Стьюдента с числом степеней свободы $df = N - 1 = 5$ в предположении приближённой нормальности выборочного среднего.

Таблица 4.2: Параметры запуска бенчмарков по уровням нагрузки

Бенчмарк	лёгкая	умеренная	стрессовая
hackbench	-l 10000	-g 4 -l 120000	-g 10 -l 50000
sysbench (поток)	1	4	20
schbench (-m/-t)	1 / 5	2 / 20	4 / 40

Набор бенчмарков отражает три качественно различных профиля нагрузки. Hackbench создаёт массовое порождение короткоживущих задач и интенсивный обмен сообщениями, проверяя механизмы пробуждения и балансировки. Sysbench воспроизводит серверный режим, в котором потоки регулярно блокируются на ожидании ответа базы данных. Schbench сочетает длительные вычислительные кванты с короткими фазами сна и измеряет задержку реакции на пробуждение. Подобное сочетание позволяет одновременно оценить способность планировщика поддерживать высокий темп переключений и минимизировать задержки на гетерогенном оборудовании.

4.3 Измеряемые метрики

Полный перечень измеряемых величин приведён в таблицах 4.3–4.5. В столбце «Интерпретация» пометка «больше лучше» обозначает метрики, у которых большее значение соответствует лучшему поведению планировщика, «меньше лучше» — метрики, у которых лучшему поведению соответствует меньшее значение, «неоднозначно» — диагностические показатели без однозначной интерпретации.

Hackbench измеряет общее время выполнения теста. Sysbench регистрирует число выполненных транзакций PostgreSQL в секунду (TPS) и число SQL-запросов в секунду (QPS). Schbench возвращает 99-й и 99.9-й процентиля двух распределений — задержки пробуждения рабочего потока и задержки обслуживания запроса, а также число

Таблица 4.3: Метрики бенчмарков

Источник	Метрика	Единица	Интерпретация
hackbench	время выполнения	с	меньше лучше
sysbench	TPS	тр./с	больше лучше
sysbench	QPS	зап./с	больше лучше
schbench	задержка пробуждения p99	мкс	меньше лучше
schbench	задержка пробуждения p99.9	мкс	меньше лучше
schbench	задержка запроса p99	мкс	меньше лучше
schbench	задержка запроса p99.9	мкс	меньше лучше
schbench	RPS	зап./с	больше лучше

запросов, обслуженных в секунду (RPS).

Таблица 4.4: Категории задержек, снимаемые инструментом `sched_latency`

Метрика	Единица	Интерпретация
schedule delay	мкс	меньше лучше
runqueue latency	мкс	меньше лучше
wakeup latency	мкс	меньше лучше
preemption latency	мкс	меньше лучше
migration latency	мкс	меньше лучше

Schedule delay — интервал от пробуждения задачи до начала её исполнения на процессоре. Runqueue latency — время, проведённое задачей в очереди готовых. Wakeup latency — задержка между запросом пробуждения и постановкой задачи в очередь. Preemption latency — задержка вытеснения исполняющейся задачи. Migration latency — задержка миграции задачи между ядрами.

Таблица 4.5: Системные счётчики и показания подсистемы Intel RAPL

Источник	Метрика	Единица	Интерпретация
/proc/stat	загрузка процессора	%	неоднозначно
/proc/schedstat	переключения контекста	шт./с	неоднозначно
/proc/schedstat	кванты в секунду	шт./с	неоднозначно
Intel RAPL	потреблённая энергия	Дж	меньше лучше

Из `/proc/stat` считывается доля времени, проведённого процессором вне состояния простоя. Из `/proc/schedstat` извлекаются частота переключений контекста и число квантов планирования в секунду. Из подсистемы Intel RAPL читается счётчик потреблённой энергии домена `package` (весь процессорный чип) в джоулях. Для каждого планировщика фиксируется приращение счётчика за время прогона, что позволяет сравнивать суммарные энергозатраты.

5 Анализ результатов эксперимента

5.1 Результаты при лёгкой нагрузке

В режиме лёгкой нагрузки каждый бенчмарк запускается с минимальными параметрами параллелизма, при которых число активных задач не превышает числа доступных ядер. Такой режим позволяет оценить качество решений планировщика в условиях, когда конкуренция за ядра минимальна и определяющим фактором становится выбор типа ядра для каждой задачи.

Результаты бенчмарка `hackbench` представлены в таблице 5.1.

Таблица 5.1: `hackbench`, лёгкая нагрузка (время выполнения, секунды)

Планировщик	Среднее	Ст. откл.	95%-ДИ
<code>scx_EEVDF</code>	1,838	0,034	[1,752; 1,923]
<code>scx_A1349</code>	1,859	0,007	[1,852; 1,867]
штатный <code>EEVDF</code>	1,900	0,012	[1,870; 1,931]
<code>LAVD</code>	1,981	0,012	[1,953; 2,010]

Планировщик `scx_EEVDF` показывает наименьшее среднее время выполнения (1,838 с), однако его доверительный интервал пересекается с интервалом `A1349` (1,859 с), что не позволяет утверждать статистически значимое различие между ними. При этом верхняя граница интервала `A1349` (1,867) меньше нижней границы `EEVDF` (1,870), что подтверждает значимое преимущество `A1349` над штатным планировщиком в 2,2%. Минимальное стандартное отклонение `A1349` ($\sigma = 0,007$) среди всех планировщиков указывает на высокую стабильность результатов.

Результаты `sysbench` приведены в таблице 5.2.

Таблица 5.2: `sysbench`, лёгкая нагрузка (транзакции в секунду)

Планировщик	Среднее TPS	Ст. откл.	95%-ДИ
<code>scx_A1349</code>	4263	1082	[3128; 5399]
штатный <code>EEVDF</code>	4246	18	[4201; 4292]
<code>LAVD</code>	4205	36	[4116; 4294]
<code>scx_EEVDF</code>	3793	201	[3295; 4292]

На `sysbench` при лёгкой нагрузке все четыре планировщика показывают сопоставимые результаты: доверительные интервалы пересекаются. `A1349` достигает среднего TPS 4263, что номинально превышает штатный `EEVDF` (4246), однако высокое стандартное отклонение ($\sigma = 1082$) не позволяет считать различие значимым. Широкий интервал обусловлен нестабильностью аукционного механизма при малом числе потоков, где смешанный профиль нагрузки `sysbench` затрудняет оптимальное назначение.

Результаты schbench приведены в таблице 5.3.

Таблица 5.3: schbench, лёгкая нагрузка (задержка пробуждения, мкс)

Планировщик	p50	p99	p99.9	Ср. RPS
scx_EEVDF	2	5	59	1668
штатный EEVDF	15	152	185	591
scx_A1349	29	163	195	2059
LAVD	38	170	208	678

По пропускной способности (средний RPS) планировщик A1349 превосходит все остальные: 2059 запросов в секунду, что в 3,5 раза выше штатного EEVDF (591) и на 23% выше **scx_EEVDF** (1668). Медианная задержка пробуждения A1349 (29 мкс) выше, чем у **scx_EEVDF** (2 мкс), но сопоставима с EEVDF (15 мкс). Хвостовые задержки p99 и p99.9 у A1349, EEVDF и LAVD близки (152–208 мкс), тогда как **scx_EEVDF** удерживает исключительно низкие хвосты (5 и 59 мкс). Высокая пропускная способность A1349 при умеренных задержках объясняется тем, что при низкой нагрузке аукционный механизм направляет вычислительные фазы schbench на Р-ядра, сокращая длительность каждого запроса.

5.2 Результаты при стрессовой нагрузке

В режиме стрессовой нагрузки число активных задач многократно превышает число доступных ядер, что создаёт интенсивную конкуренцию за вычислительные ресурсы. Данный режим позволяет оценить работу планировщика в условиях перегрузки, где критически важны справедливость распределения и эффективность балансировки между кластерами ядер.

Результаты haskbench при стрессовой нагрузке представлены в таблице 5.4.

Таблица 5.4: haskbench, стрессовая нагрузка (время выполнения, секунды)

Планировщик	Среднее	Ст. откл.	95%-ДИ
scx_EEVDF	9,176	0,072	[8,998; 9,354]
scx_A1349	9,283	0,021	[9,262; 9,305]
штатный EEVDF	9,344	0,062	[9,189; 9,498]
LAVD	9,732	0,079	[9,536; 9,928]

При стрессовой нагрузке **scx_EEVDF** и **scx_A1349** опережают штатный EEVDF, LAVD от него отстаёт. Планировщик **scx_EEVDF** показывает наименьшее время (9,176 с), за ним следует A1349 (9,283 с). Доверительный интервал A1349 [9,262; 9,305] полностью укладывается в интервал **scx_EEVDF** [8,998; 9,354], поэтому различие между ними не является статистически значимым. Стандартное отклонение A1349 ($\sigma = 0,021$) минимально среди всех планировщиков, что свидетельствует о стабильности при высокой нагрузке.

Результаты sysbench при стрессовой нагрузке приведены в таблице 5.5.

Таблица 5.5: sysbench, стрессовая нагрузка (транзакции в секунду)

Планировщик	Среднее TPS	Ст. откл.	95%-ДИ
штатный EEVDF	67 373	939	[65 042; 69 705]
LAVD	51 244	1060	[48 610; 53 877]
scx_A1349	44 097	744	[43 316; 44 878]
scx_EEVDF	41 803	998	[39 324; 44 282]

На sysbench при стрессовой нагрузке штатный EEVDF значительно опережает все sched_ext планировщики. Планировщик A1349 достигает 65,5% пропускной способности EEVDF (44 097 TPS против 67 373 TPS), но опережает scx_EEVDF (41 803 TPS) на 5,5%. Отставание sched_ext планировщиков объясняется двумя факторами: накладными расходами на BPF-вызовы при высоком темпе переключений контекста и отсутствием интеграции с механизмом `wake_affine`, оптимизирующим локальность кэша при пробуждении.

Результаты schbench при стрессовой нагрузке представлены в таблице 5.6.

Таблица 5.6: schbench, стрессовая нагрузка (задержка пробуждения, мкс)

Планировщик	p50	p99	p99.9	Ср. RPS
scx_A1349	3056	10 240	16 061	8654
LAVD	3735	20 789	59 872	166
штатный EEVDF	4589	11 131	17 163	5645
scx_EEVDF	9147	16 112	16 672	8639

При стрессовой нагрузке планировщик A1349 демонстрирует наименьшую медианную задержку пробуждения 3056 мкс, что на 18% ниже ближайшего конкурента LAVD (3735 мкс) и на 33% ниже штатного EEVDF (4589 мкс). По процентилям p99 и p99.9 A1349 также лидирует: 10,2 мс и 16,1 мс соответственно. Средний RPS составляет 8654, что на 53% выше EEVDF (5645) и сопоставимо с scx_EEVDF (8639). Аукционный механизм эффективно распределяет задачи между кластерами при перегрузке: латентно-чувствительные задачи получают Р-ядра, а длительные вычисления перенаправляются на Е-ядра.

Латентность обслуживания запросов schbench представлена в таблице 5.7.

Таблица 5.7: schbench, стрессовая нагрузка (задержка запроса, мкс)

Планировщик	p50	p99	p99.9
штатный EEVDF	6643	444 245	1 670 485
LAVD	7587	37 464 747	38 207 488
scx_EEVDF	8763	22 347	24 907
scx_A1349	10 240	17 685	22 837

По задержке обслуживания запросов A1349 показывает наилучшие хвостовые показатели: p99 составляет 17,7 мс, а p99.9 — 22,8 мс. Штатный EEVDF при стрессовой нагрузке демонстрирует катастрофическую деградацию: p99 достигает 444 мс, а p99.9 — 1,67 с. LAVD показывает ещё худшие результаты с задержками порядка десятков секунд. Данное поведение указывает на то, что при перегрузке штатный EEVDF и LAVD допускают голодание отдельных задач, тогда как бюджетные ограничения и VCG-платежи A1349 обеспечивают более равномерное распределение ресурсов.

5.3 Результаты при умеренной нагрузке

Режим умеренной нагрузки занимает промежуточное положение: число активных задач сопоставимо с числом ядер, но часть из них выполняется одновременно, создавая умеренную конкуренцию за ресурсы.

Результаты `hackbench` при умеренной нагрузке представлены в таблице 5.8.

Таблица 5.8: `hackbench`, умеренная нагрузка (время выполнения, секунды)

Планировщик	Среднее	Ст. откл.	95%-ДИ
<code>sctx_EEVDF</code>	8,716	0,031	[8,640; 8,793]
<code>sctx_A1349</code>	9,030	0,014	[9,015; 9,045]
штатный EEVDF	9,655	0,011	[9,628; 9,683]
LAVD	10,121	0,039	[10,023; 10,218]

Планировщик A1349 занимает второе место после `sctx_EEVDF`, опережая штатный EEVDF на 6,5% (9,030 с против 9,655 с). Доверительные интервалы A1349 и EEVDF не пересекаются, что подтверждает статистическую значимость различия. Стандартное отклонение A1349 ($\sigma = 0,014$) указывает на стабильное поведение аукционного механизма при умеренной конкуренции.

Результаты `sysbench` при умеренной нагрузке приведены в таблице 5.9.

Таблица 5.9: `sysbench`, умеренная нагрузка (транзакции в секунду)

Планировщик	Среднее TPS	Ст. откл.	95%-ДИ
штатный EEVDF	16 566	32	[16 487; 16 645]
<code>sctx_A1349</code>	15 298	1829	[13 379; 17 217]
LAVD	14 233	17	[14 191; 14 276]
<code>sctx_EEVDF</code>	12 517	96	[12 279; 12 756]

На `sysbench` планировщик A1349 занимает второе место с 15 298 TPS, уступая штатному EEVDF на 7,7%. Широкий доверительный интервал A1349 ([13 379; 17 217]) пересекается с интервалом EEVDF, что не позволяет считать различие значимым. Средний результат A1349 превышает LAVD на 7,5% и `sctx_EEVDF` на 22,2%, что подтверждает положительное влияние учёта гетерогенности при обработке смешанных нагрузок.

Результаты `schbench` при умеренной нагрузке представлены в таблице 5.10.

Таблица 5.10: schbench, умеренная нагрузка (задержка пробуждения, мкс)

Планировщик	p50	p99	p99.9	Ср. RPS
LAVD	5	881	1437	4281
<code>scx_EEVDF</code>	23	2375	2815	8585
штатный EEVDF	85	3300	4733	5700
<code>scx_A1349</code>	743	3331	9744	8638

На schbench при умеренной нагрузке планировщик A1349 достигает наивысшей пропускной способности: 8638 RPS, что на 51,5% выше штатного EEVDF (5700) и сопоставимо с `scx_EEVDF` (8585). Однако медианная задержка пробуждения A1349 составляет 743 мкс, что значительно выше, чем у LAVD (5 мкс), `scx_EEVDF` (23 мкс) и EEVDF (85 мкс). Хвостовая задержка p99.9 у A1349 достигает 9,7 мс. Повышенные задержки объясняются тем, что при умеренной конкуренции бюджетный механизм расходует токены быстрее, чем они восстанавливаются, вынуждая часть задач ожидать в очереди STARVED с пониженным приоритетом. При этом высокая пропускная способность указывает на эффективную утилизацию ядер обоих типов: задачи обслуживаются быстрее благодаря назначению на подходящий тип ядра.

5.4 Выводы

Проведённый эксперимент позволяет сформулировать основные закономерности поведения планировщика A1349 на гетерогенной платформе в сравнении со штатным EEVDF, его повторной реализацией на sched_ext (`scx_EEVDF`) и планировщиком LAVD.

Преимущества модели A1349. На бенчмарке haskbench планировщик A1349 стабильно опережает штатный EEVDF на всех уровнях нагрузки: на 2,2% при лёгкой, на 6,5% при умеренной и сопоставим при стрессовой нагрузке. При этом стандартное отклонение A1349 минимально или близко к минимальному во всех режимах, что указывает на предсказуемость поведения. Результат объясняется тем, что аукционный механизм направляет короткоживущие задачи haskbench преимущественно на Р-ядра, где они завершаются быстрее.

По пропускной способности schbench (RPS) планировщик A1349 лидирует на всех уровнях нагрузки: в 3,5 раза выше штатного EEVDF при лёгкой, на 51,5% при умеренной и на 53% при стрессовой нагрузке. При стрессовой нагрузке A1349 лидирует и по всем перцентилям задержки пробуждения: p50 = 3056 мкс (на 33% ниже EEVDF), p99 = 10,2 мс и p99.9 = 16,1 мс. Это объясняется свойством модели: задачи с высокой эффективной ценностью $\phi_P(\theta_i)$ (чувствительные к задержке) получают Р-ядра по результатам аукциона, что сокращает задержку пробуждения и длительность обслуживания.

Ограничения модели A1349. На бенчмарке sysbench планировщик A1349 уступает штатному EEVDF: от статистически незначимого различия при лёгкой нагрузке до 34,5% при стрессовой. Аналогичную деградацию демонстрируют все

`sched_ext` планировщики, что указывает на системные накладные расходы фреймворка при нагрузках с высоким темпом переключений контекста. Дополнительным фактором является отсутствие в `sched_ext` встроенной интеграции с механизмом `wake_affine`, оптимизирующим локальность кэша при пробуждении.

При умеренной нагрузке `schbench` планировщик A1349 показывает наихудшую медианную задержку пробуждения (743 мкс против 5–85 мкс у конкурентов). Данный режим выявляет особенность бюджетного механизма: при умеренной конкуренции бюджеты расходуются быстрее, чем восстанавливаются, вынуждая задачи ожидать в очереди с пониженным приоритетом. При этом пропускная способность A1349 остаётся наивысшей (8638 RPS), что указывает на эффективную утилизацию ядер обоих типов несмотря на повышенные задержки отдельных пробуждений.

Влияние свойств модели на результаты. Учёт гетерогенности (параметр σ модели) проявляется в стабильном превосходстве по пропускной способности `schbench` и сокращении времени `hackbench`: зная соотношение производительностей ядер, механизм назначает латентно-чувствительные задачи на P-ядра и фоновые — на E-ядра, что невозможно в рамках штатного EEVDF, оперирующего единым виртуальным временем для всех ядер. VCG-распределение обеспечивает приоритет задачам с наибольшей эффективной ценностью без явного задания приоритетов, а бюджетные ограничения B_i предотвращают монополизацию P-ядер, что подтверждается хвостовыми задержками обслуживания запросов при стрессовой нагрузке: 22,8 мс у A1349 против 1,67 с у EEVDF в p99.9.

Ограничения эксперимента. Все планировщики тестировались с $n = 6$ прогонами. Эксперимент проведён на единственной аппаратной платформе (Intel Arrow Lake-S), и переносимость результатов на архитектуры ARM big.LITTLE и RISC-V требует дополнительной проверки. Кроме того, использованные бенчмарки не покрывают нагрузки реального времени и GPU-ориентированные сценарии, для которых поведение модели может существенно отличаться.

Заключение

В работе предложен и реализован планировщик для гетерогенных процессорных архитектур, проведена его экспериментальная оценка на реальной аппаратной платформе. Выполнен обзор подходов к реализации планировщиков в современных операционных системах и обоснован выбор фреймворка `sched_ext` как наиболее подходящего инструмента для решения поставленной задачи.

Построена математическая модель A1349, основанная на динамическом VCG-механизме (Викри–Кларка–Гровса) и трактующая выбор ядра для исполнения задачи как исход аукциона. Задача описывается ценностью и требуемой длительностью. Эффективная ценность задачи на ядре учитывает стоимость кванта этого ядра, что отражает разницу производительности P/E ядер. Аукцион назначает задачу на тот тип ядра, где её эффективная ценность максимальна, VCG-платёж задаёт цену вытеснения конкурента, а бюджетные ограничения предотвращают монополизацию производительных ядер и обеспечивают пропорциональный доступ без явного задания приоритетных классов.

Предложенная модель и алгоритм EEVDF реализованы средствами `sched_ext`, что позволило сравнить A1349 со штатным планировщиком ядра. Разработан воспроизводимый набор бенчмарков на основе `hackbench`, `sysbench` и `schbench`, охватывающий три уровня нагрузки (лёгкую, умеренную и стрессовую), и проведён эксперимент на гетерогенной платформе Intel Arrow Lake-S с P/E ядрами.

Полученные результаты показывают, что A1349 стабильно превосходит штатный EEVDF на нагрузках, чувствительных к размещению задач по типам ядер. На `hackbench` выигрыш составляет до 6,5% при умеренной нагрузке при минимальном стандартном отклонении. На `schbench` пропускная способность A1349 в 3,5 раза выше штатного EEVDF при лёгкой нагрузке и более чем на 50% выше при умеренной и стрессовой. В стрессовом режиме A1349 одновременно лидирует по медианной и хвостовой задержке пробуждения. На `sysbench` все `sched_ext` планировщики уступают штатному EEVDF, что указывает на накладные расходы фреймворка, а не самой модели A1349.

Из этого следует, что учёт гетерогенности на уровне математической модели даёт измеримый выигрыш на нагрузках смешанного характера, а инфраструктура `sched_ext` пригодна для прототипирования и эксплуатации специализированных политик планирования при условии, что профиль нагрузки не состоит преимущественно из коротких блокирующих ввод-вывод операций.

Перспективным направлением дальнейших исследований представляется проверка переносимости модели A1349 на процессоры ARM big.LITTLE и гетерогенные конфигурации RISC-V, а также адаптация бюджетного механизма к режимам умеренной конкуренции, в которых наблюдается рост медианной задержки пробуждения на `schbench`.

Список литературы

- [1] John Lions A Commentary on the Sixth Edition UNIX Operating System The University of New South Wales 1977
- [2] Mike Kravetz, Hubertus Franke, Shailabh Nagar, Rajan Ravindran Enhancing Linux Scheduler Scalability 2001
- [3] Robert Love Linux Kernel Development 2nd ed Novell Press, 2005
- [4] C. S. Wong, I. K. T. Tan, R. D. Kumari, J. W. Lam, W. Fun Fairness and Interactive Performance of O(1) and CFS Linux Kernel Schedulers International Symposium on Information Technology 2008
- [5] Ion Stoica, Hussein Abdel-Wahab Earliest Eligible Virtual Deadline First : A Flexible and Accurate Mechanism for Proportional Share Resource Allocation Old Dominion University 1996
- [6] Greenhalgh P. Big.LITTLE Processing with ARM Cortex-A15 & Cortex-A7 ARM White Paper, 2011
- [7] Rotem E. et al. Alder Lake Architecture 2021 IEEE Hot Chips 33 Symposium (HCS), Palo Alto, CA, USA, 2021, pp. 1–23, doi: 10.1109/HCS52781.2021.9567097
- [8] Conti F., Paulin G., Garofalo A., Rossi D., Pullini A., Benini L. Marsellus: A Heterogeneous RISC-V AI-IoT End-Node SoC with 2-to-8b DNN Acceleration and 30%-Boost Adaptive Body Biasing IEEE Journal of Solid-State Circuits, 2024
- [9] Energy Aware Scheduling [Электронный ресурс]. Linux Kernel Documentation. Режим доступа: <https://www.kernel.org/doc/html/latest/scheduler/sched-energy.html> – Дата обращения 14.11.2025
- [10] Corbet J. The BPF extensible scheduler class LWN.net, 30 November 2022 Режим доступа: <https://lwn.net/Articles/916291/> – Дата обращения 20.11.2025
- [11] Humphries J. T., Natu N., Chaugule A., Weisse O., Rhoden B., Don J., Rizzo L., Rombakh O., Turner P., Kozyrakis C ghOSt: Fast & Flexible User-Space Delegation of Linux Scheduling SOSP 2021
- [12] Tang Q., Gao X., Li G., Lin J Optimizing Linux Scheduling Based on Global Runqueue with SCX IEEE International Conference SMC 2024
- [13] Xu J., Wolff D., Yi H. X., Li J., Roychoudhury A. Concurrency Testing in the Linux Kernel via eBPF arXiv 2025

- [14] Mao J., Ujcich B. E., Ma S Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [15] Wang X., Jia S., Huang Z., Cao J., Song M Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [16] Galin M., Hedblom A Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [17] Van Craeynest K., Jaleel A., Eeckhout L., Narvaez P., Emer J. S Scheduling Heterogeneous Multi-Cores through Performance Impact Estimation (PIE) ISCA, ACM, 2012, pp. 213–224
- [18] Sano R. A Dynamic Mechanism Design for Scheduling with Different Use Lengths Institute of Economic Research, Kyoto University, 2013
- [19] Leon V., Etesami S. R. Online Learning for Dynamic Vickrey-Clarke-Groves Mechanism in Unknown Environments arXiv:2506.19038, 2025
- [20] Dobzinski S., Lavi R., Nisan N. Multi-unit Auctions with Budget Limits // Games and Economic Behavior. 2012. Vol. 74, No. 2. P. 486–503. (Extended abstract: FOCS, 2008. P. 260–269.)